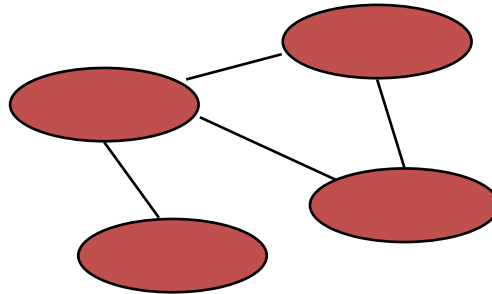


Graphs (Cont.)

graph (many to many)



Graph Searching

- Given: a graph $G = (V, E)$, directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
 - Pick a vertex as the root
 - Choose certain edges to produce a tree
 - Note: might also build a *forest* if graph is not connected
- There are two standard graph traversal techniques:
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)

Breadth-First Search

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.

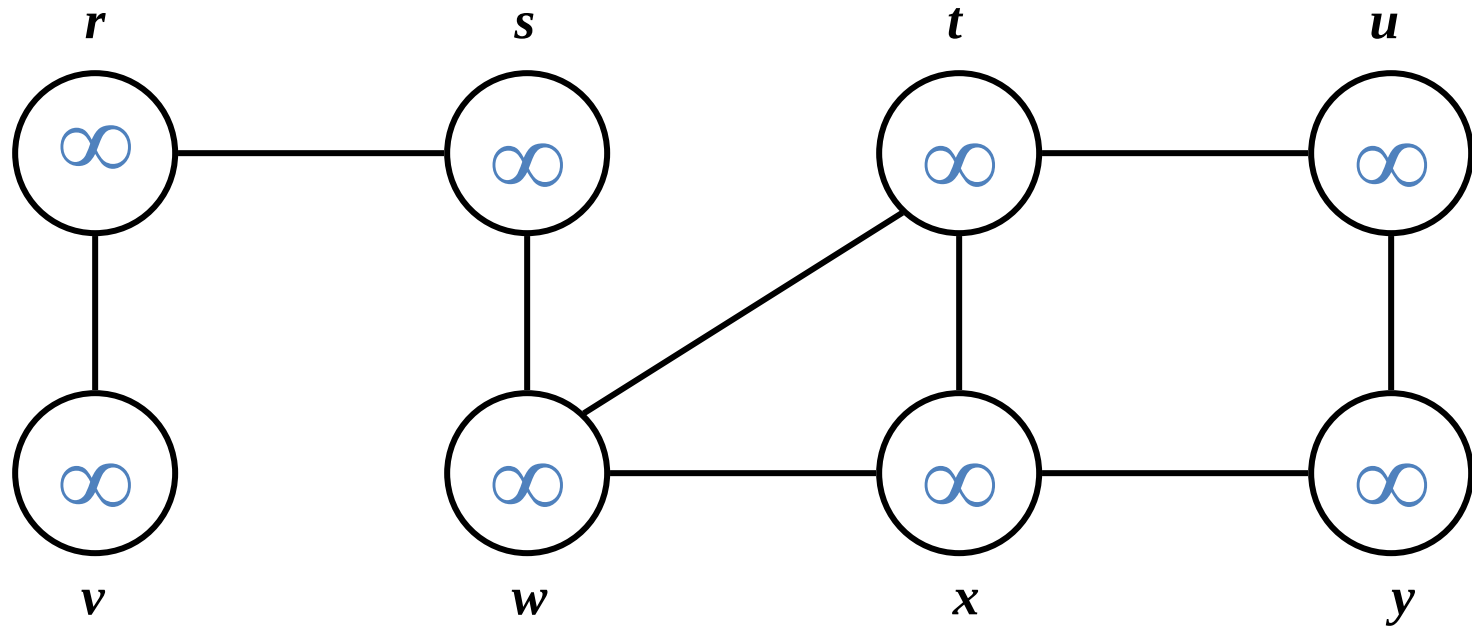
Breadth-First Search

- Again will associate vertex “colors” to guide the algorithm
 - **White vertices** have not been discovered
 - All vertices start out white
 - **Grey vertices** are discovered but not fully explored
 - They may be adjacent to white vertices
 - **Black vertices** are discovered and fully explored
 - They are adjacent only to black and grey vertices
- Explore vertices by scanning **adjacency list** of grey vertices

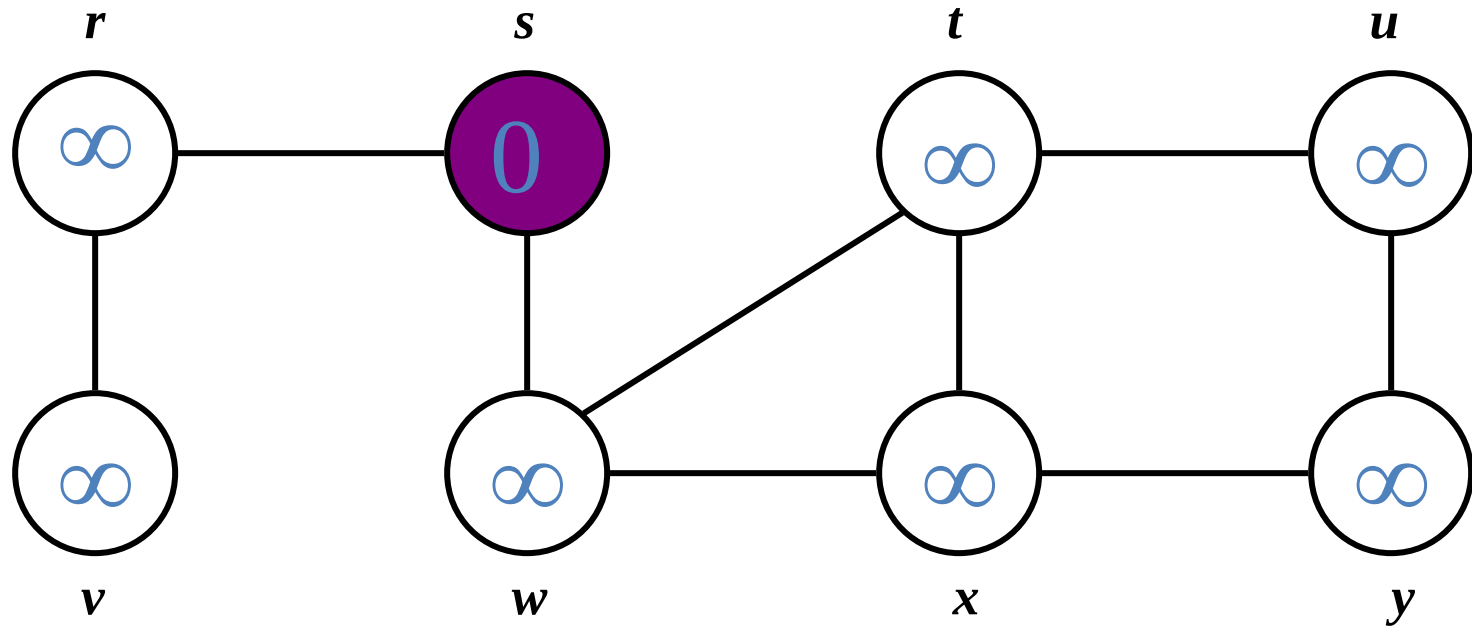
BFS(G, s)

```
1  for each vertex  $u \in V[G] - \{s\}$ 
2      do  $color[u] \leftarrow \text{WHITE}$ 
3           $d[u] \leftarrow \infty$ 
4           $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{GRAY}$ 
6   $d[s] \leftarrow 0$ 
7   $\pi[s] \leftarrow \text{NIL}$ 
8   $Q \leftarrow \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11     do  $u \leftarrow \text{DEQUEUE}(Q)$ 
12         for each  $v \in Adj[u]$ 
13             do if  $color[v] = \text{WHITE}$ 
14                 then  $color[v] \leftarrow \text{GRAY}$ 
15                      $d[v] \leftarrow d[u] + 1$ 
16                      $\pi[v] \leftarrow u$ 
17                     ENQUEUE( $Q, v$ )
18      $color[u] \leftarrow \text{BLACK}$ 
```

Breadth-First Search: Example

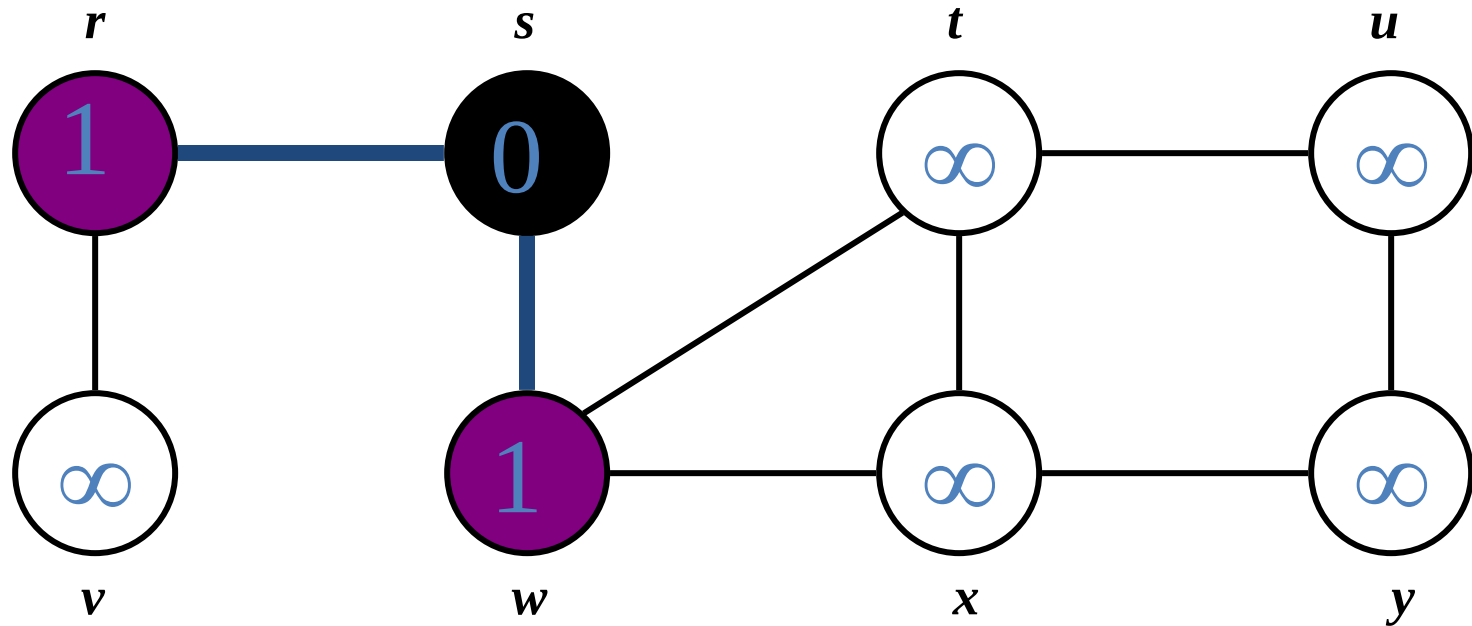


Breadth-First Search: Example



$Q:$ s

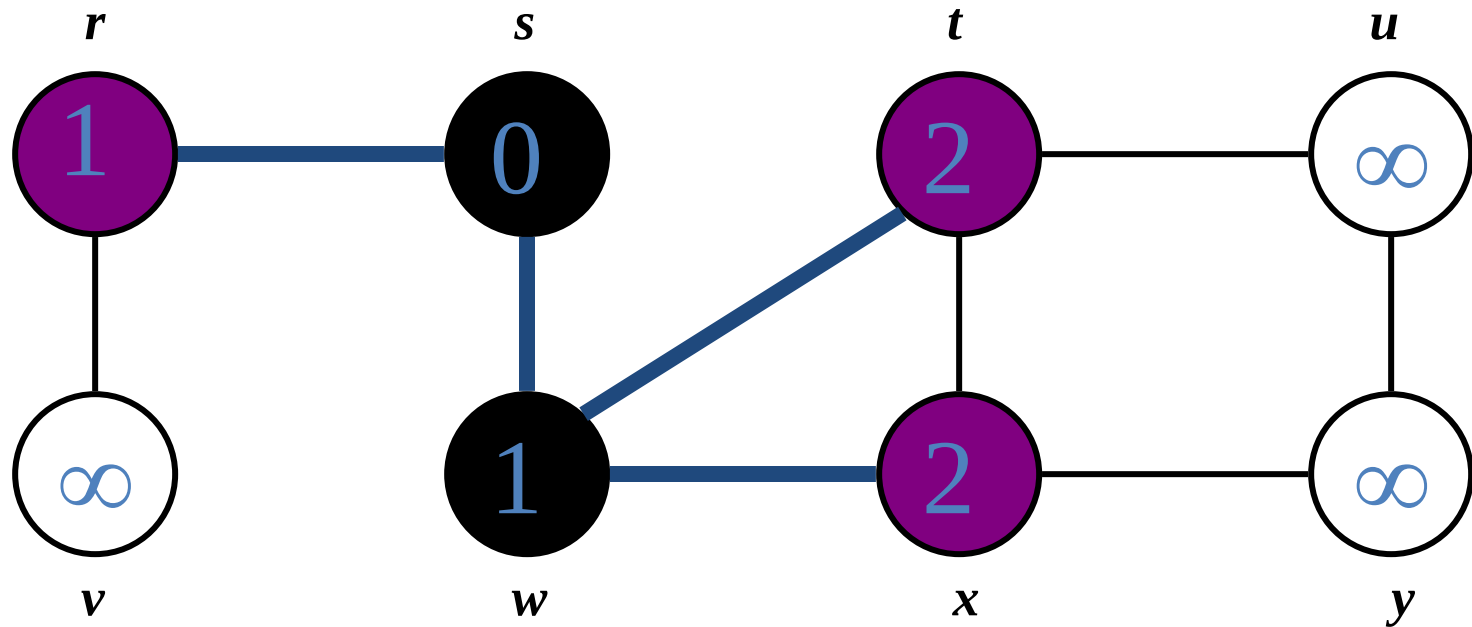
Breadth-First Search: Example



Q :

w	r
-----	-----

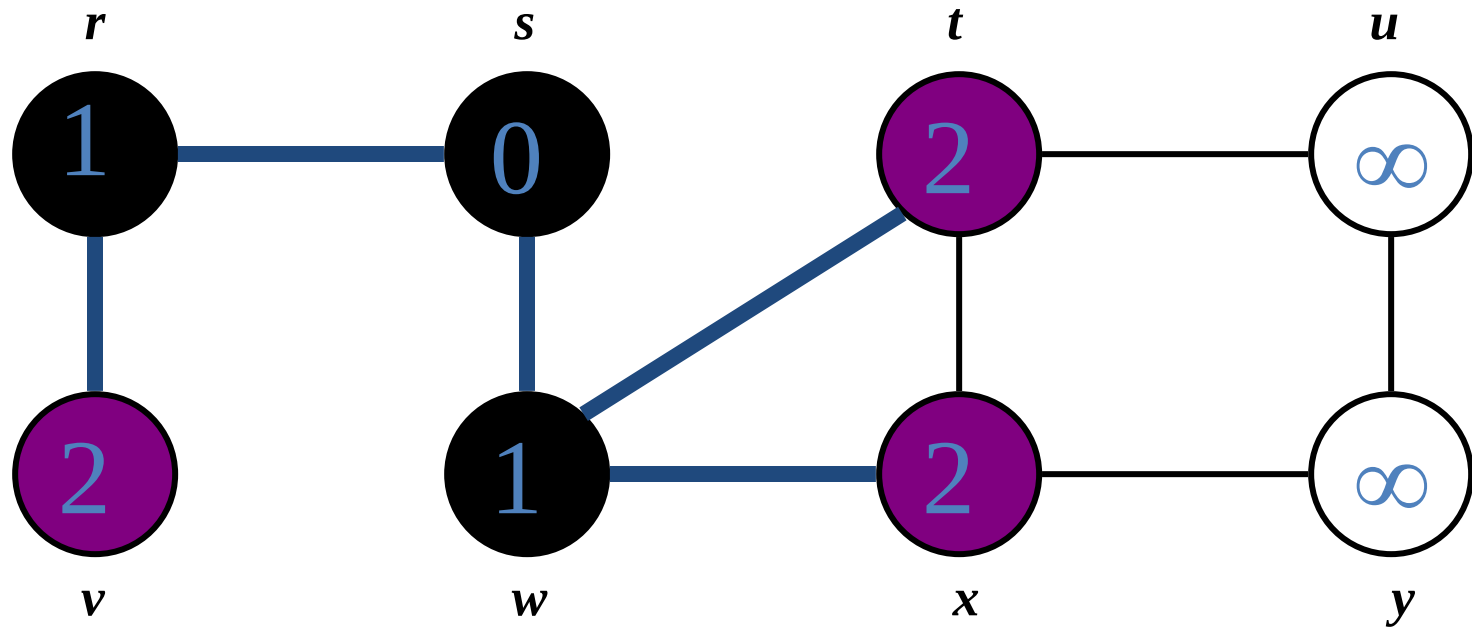
Breadth-First Search: Example



Q :

r	t	x
-----	-----	-----

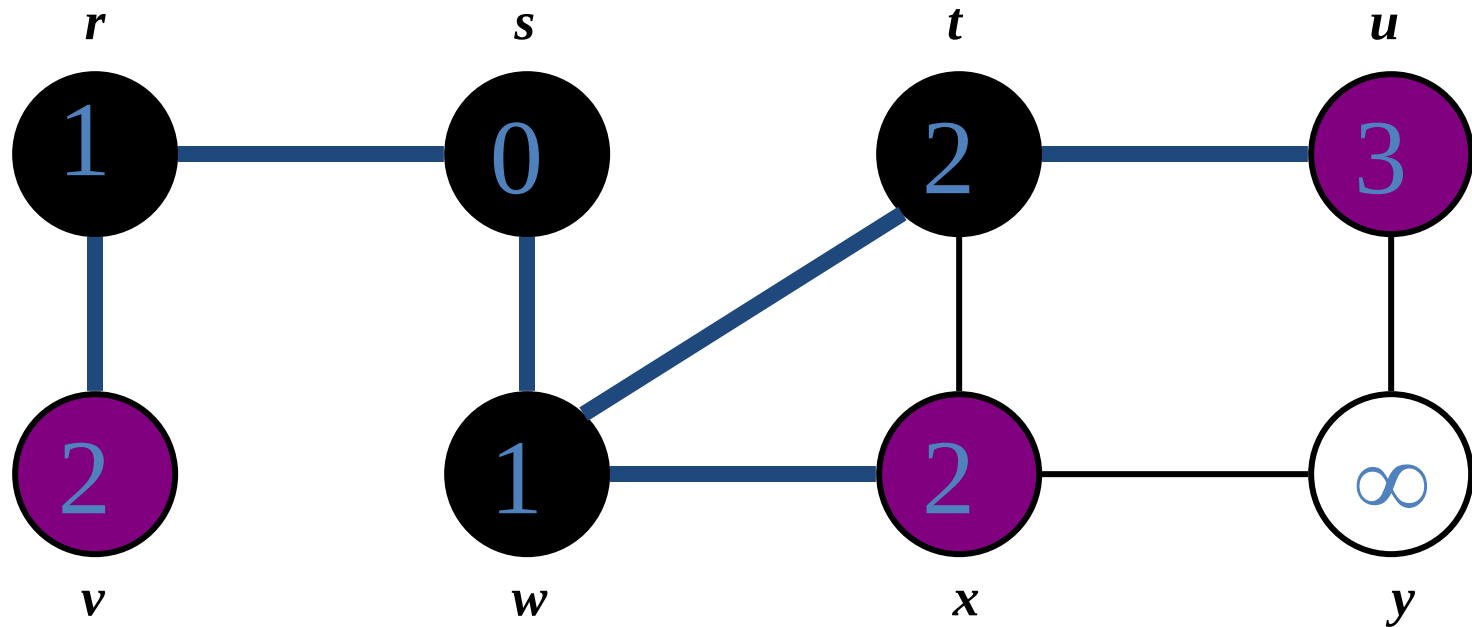
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

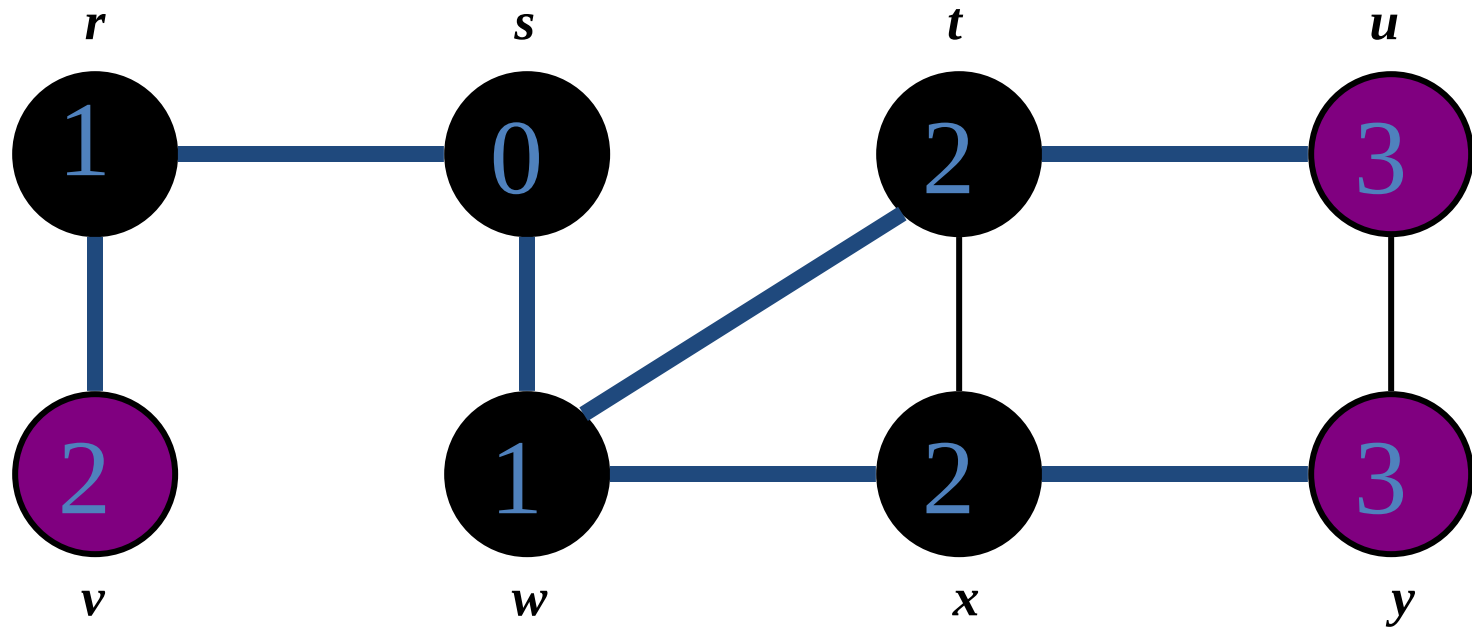
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

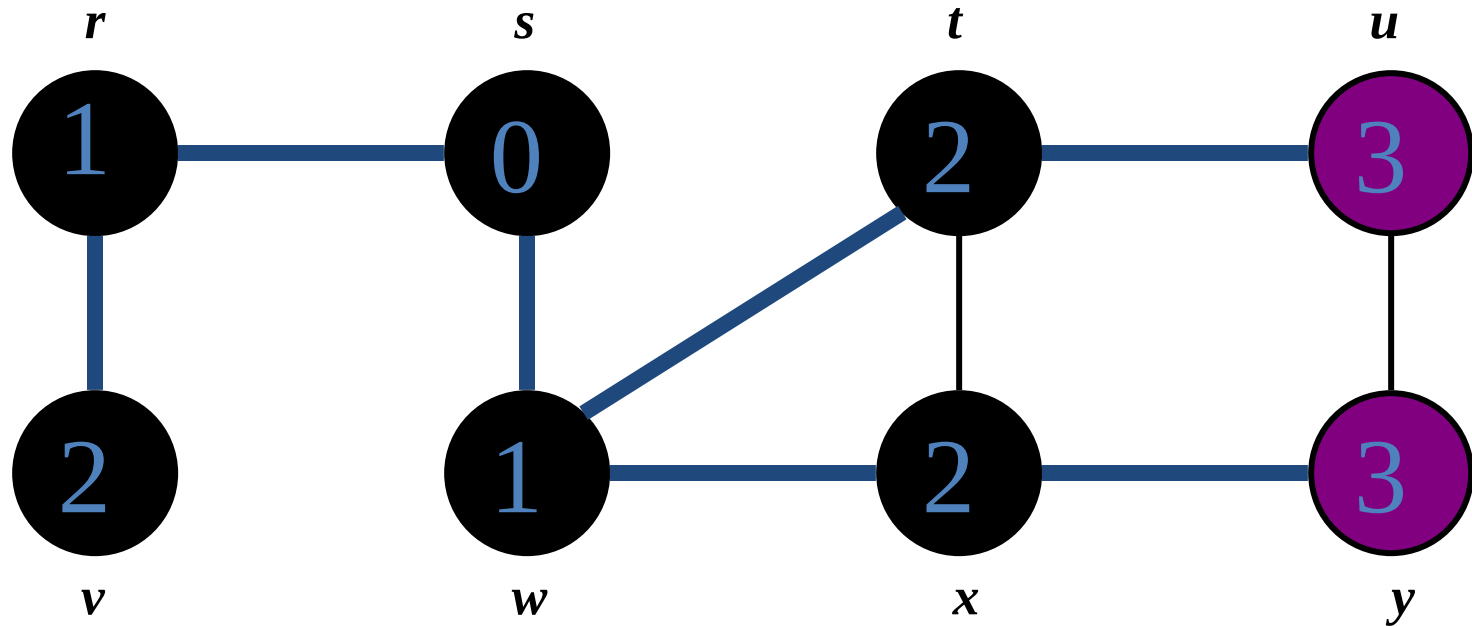
Breadth-First Search: Example



Q :

v	u	y
-----	-----	-----

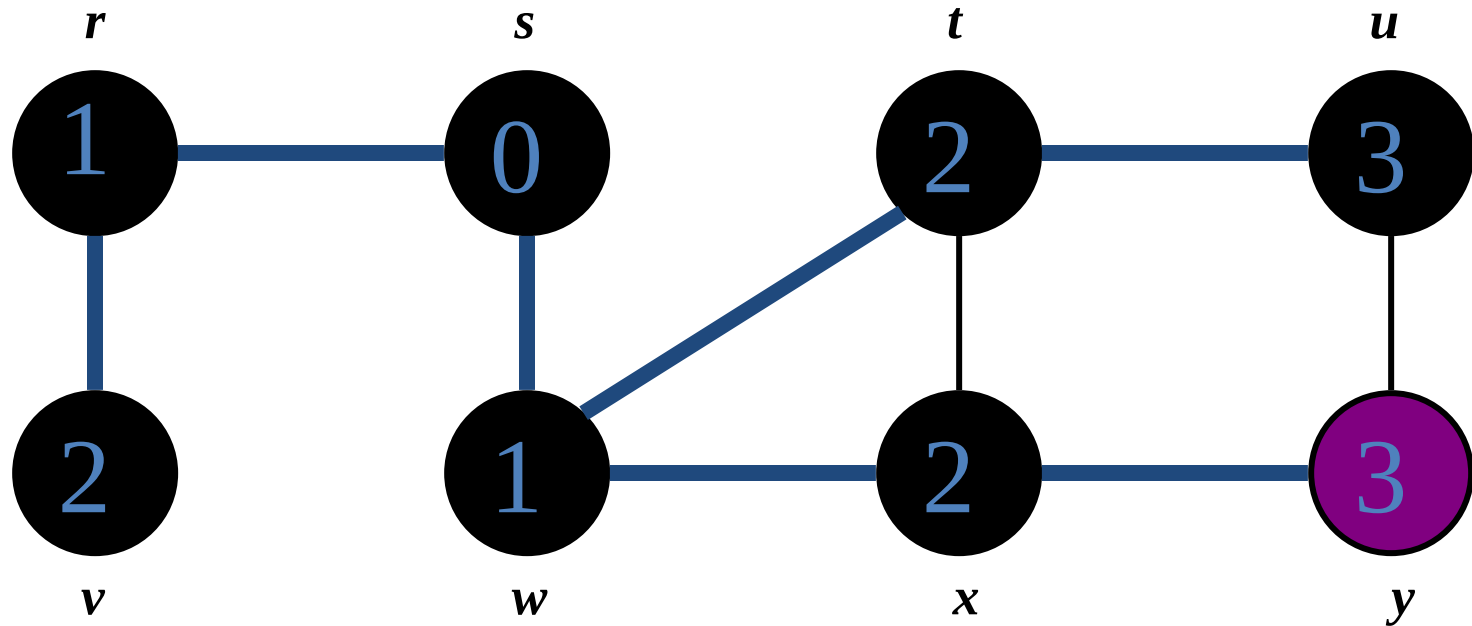
Breadth-First Search: Example



Q :

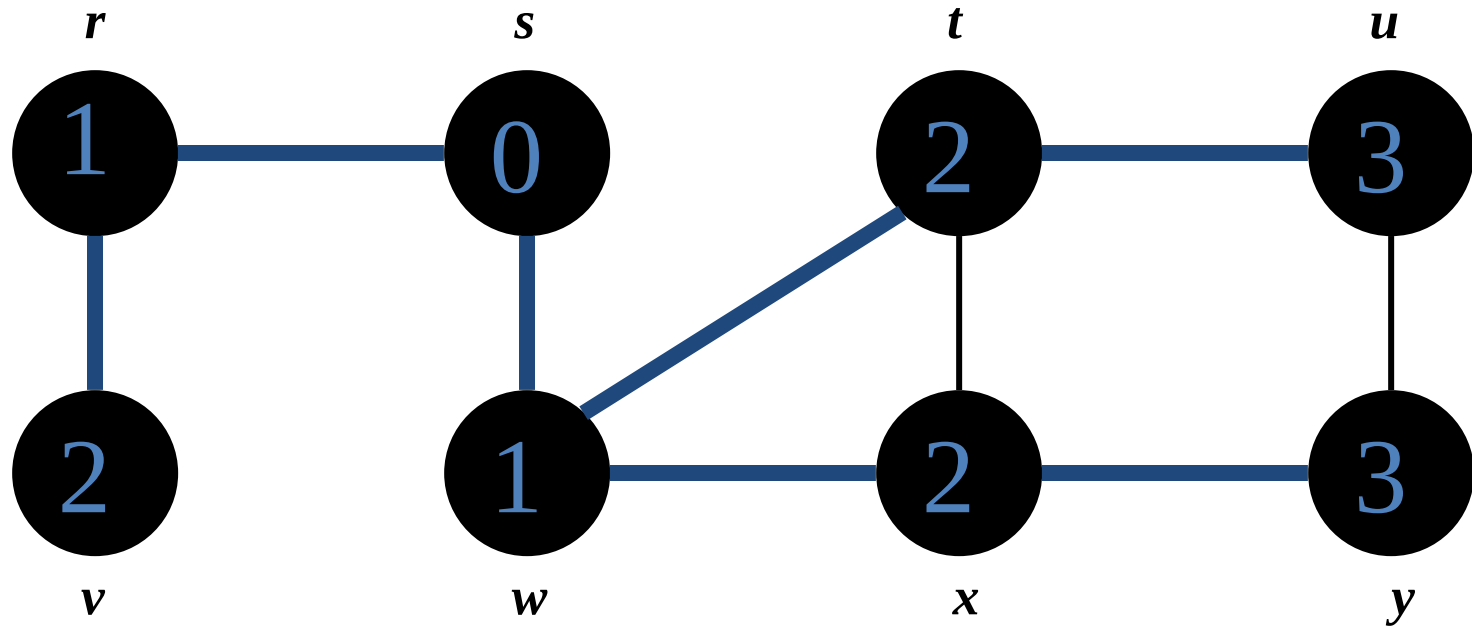
u	y
-----	-----

Breadth-First Search: Example



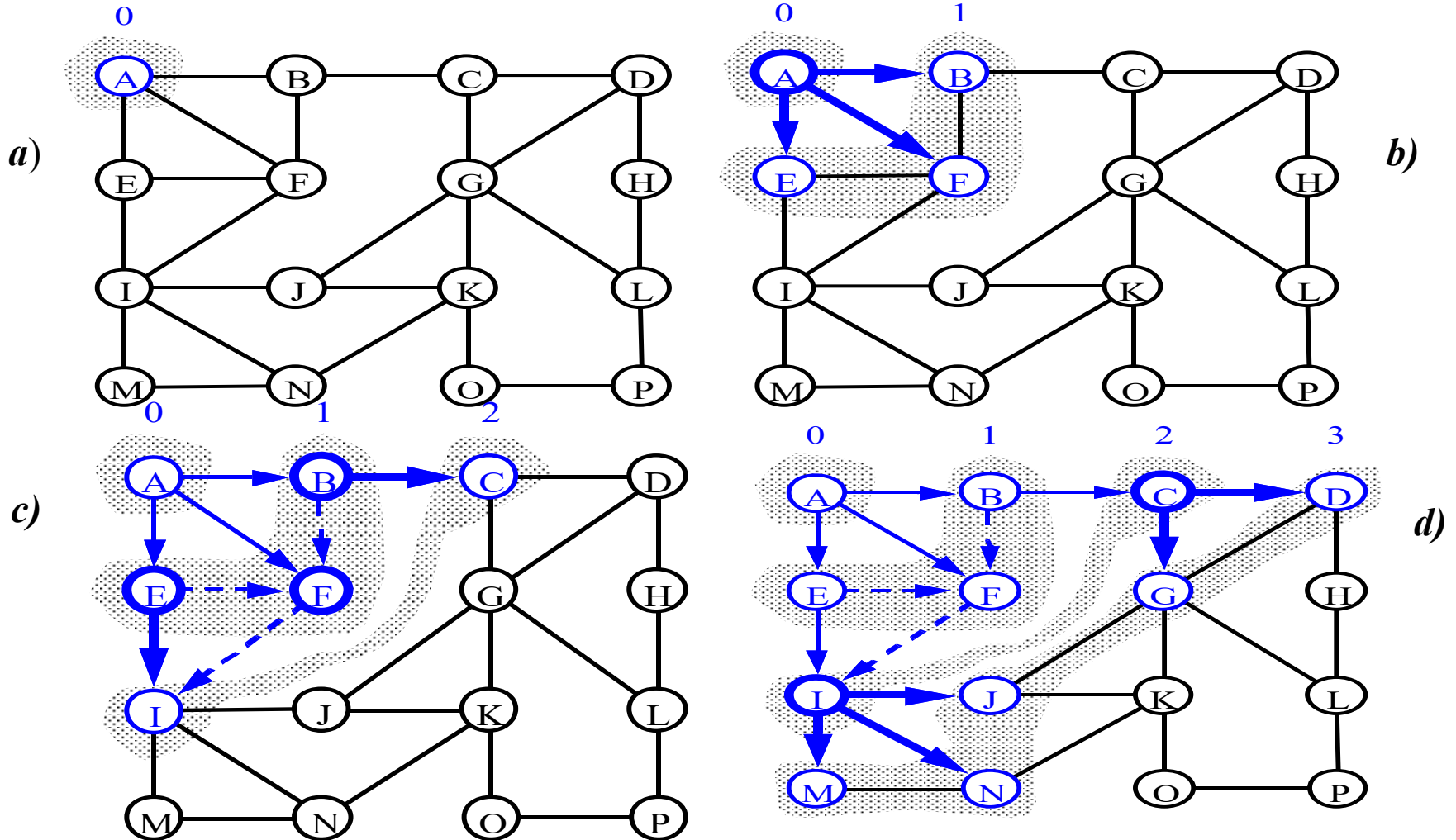
Q : y

Breadth-First Search: Example



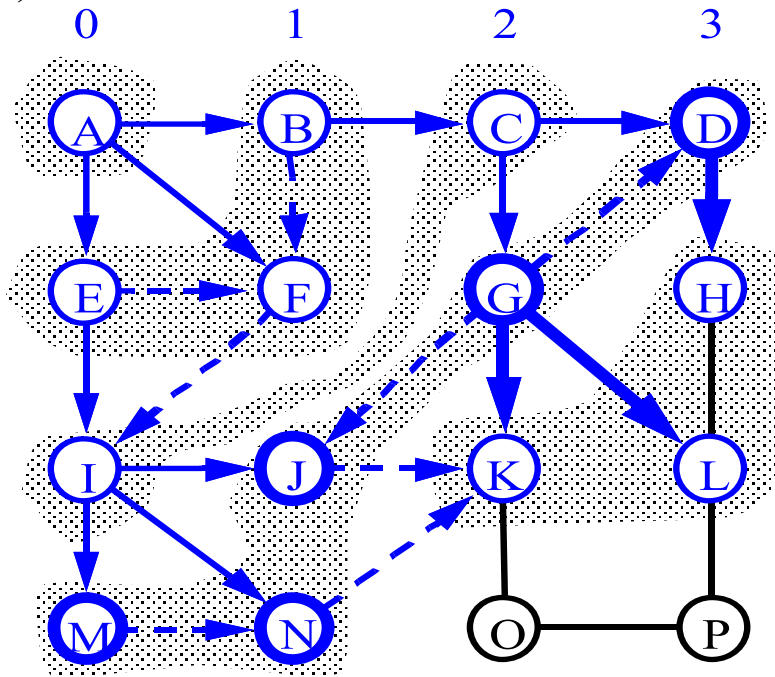
$Q: \emptyset$

BFS - A Graphical Representation

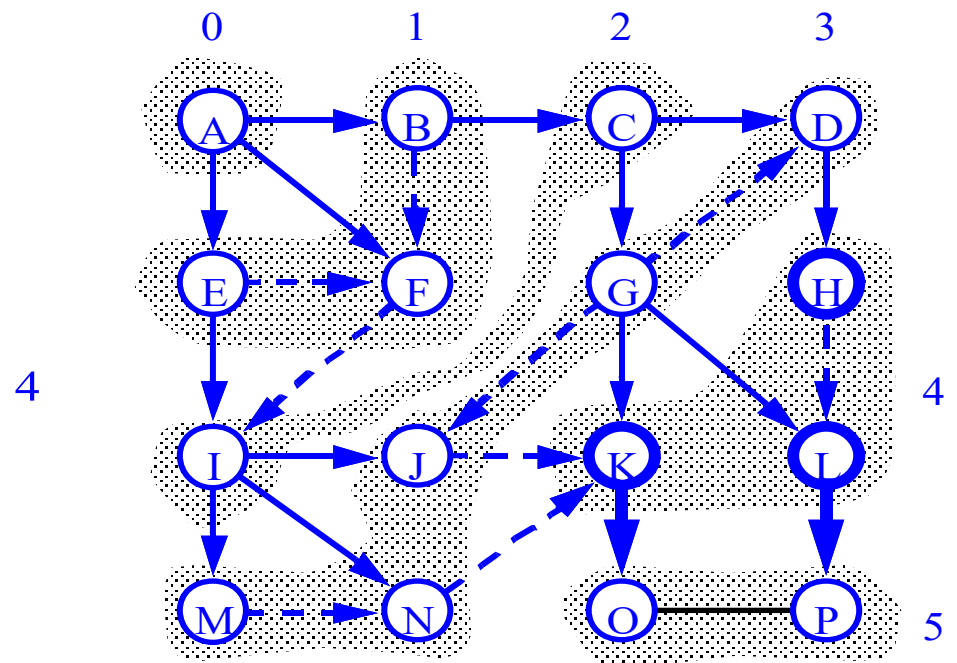


BFS - A Graphical Representation

e)



f)



BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  Touch every vertex:  $O(V)$   
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v  $\in$  u->adj {  u = every vertex, but only once  
(Why?)  
            if (v->color == WHITE)  
                v->color = GREY;  
            v->d = u->d + 1;  
            v->p = u;  
            Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

So v = every vertex that appears in some other vert's adjacency list

What will be the running time?

Total running time: $O(V+E)$

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

*What will be the storage cost
in addition to storing the tree?*

Total space used:

$$O(V + \sum(\text{degree}(v))) = O(V+E)$$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
 - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
 - Thus can use BFS to calculate shortest path from one vertex to another in $O(V+E)$ time in an unweighted tree

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
 - Explore “deeper” in the graph whenever possible
 - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
 - When all of v 's edges have been explored, backtrack to the vertex from which v was discovered
- Vertices initially colored white
- Then colored grey when discovered
- Then black when finished

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u  $\in$  G- $\rightarrow$ V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u  $\in$  G- $\rightarrow$ V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v  $\in$  u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->d represent?

Depth-First Search: The Code

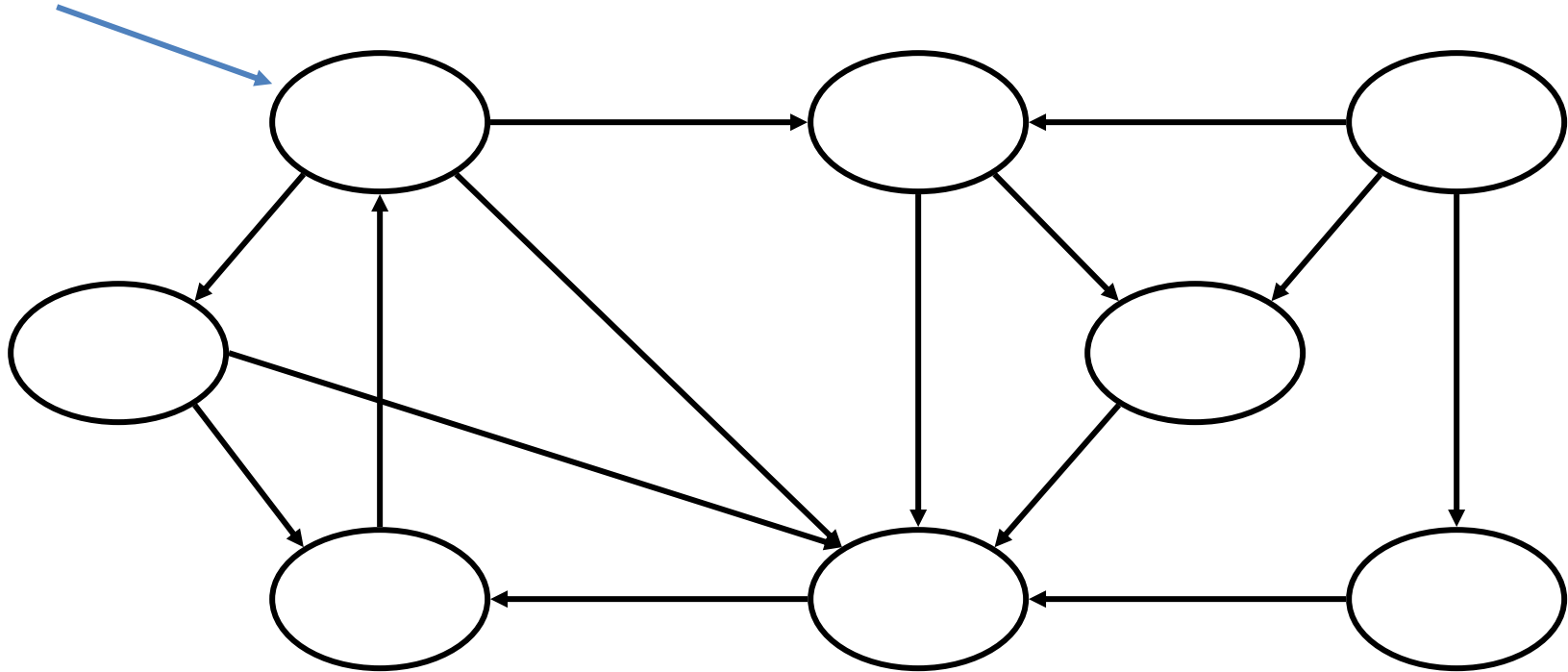
```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->f represent?

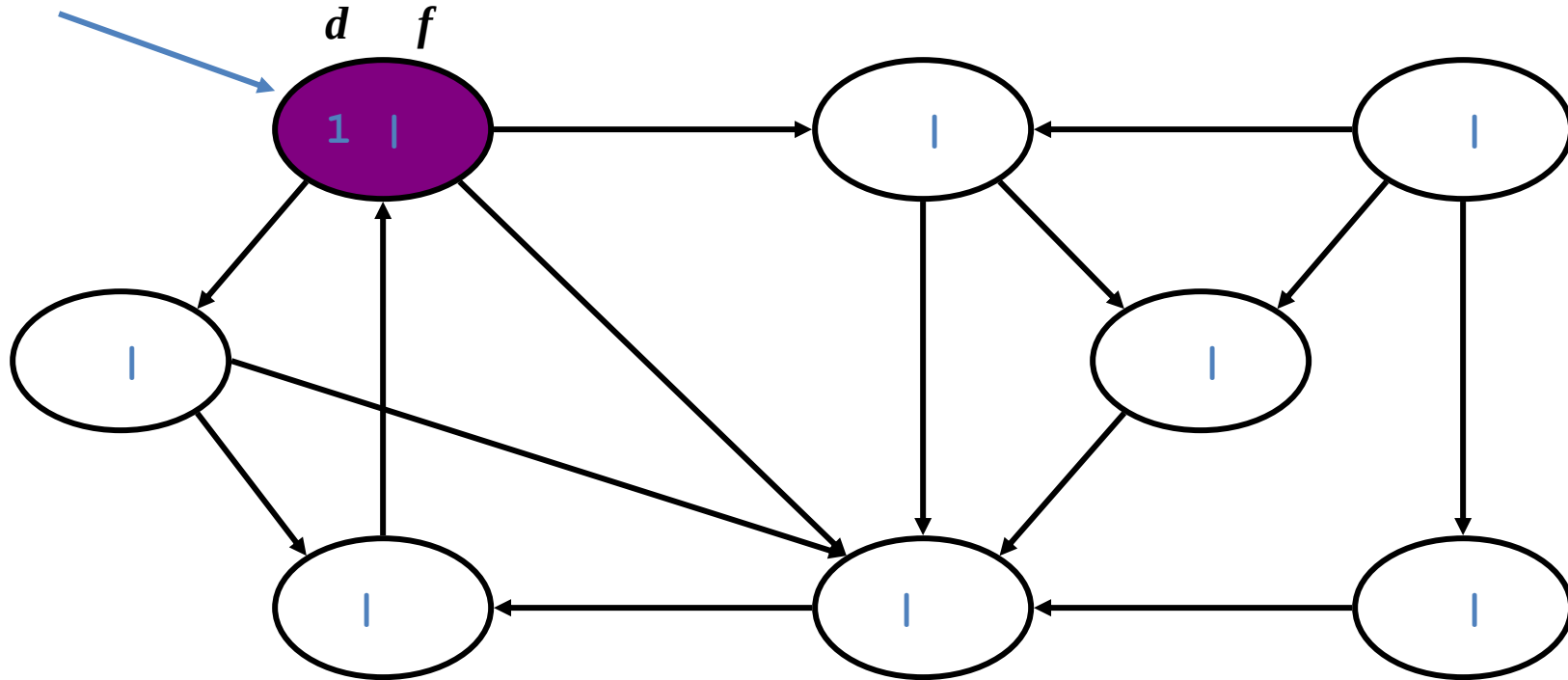
DFS Example

*source
vertex*



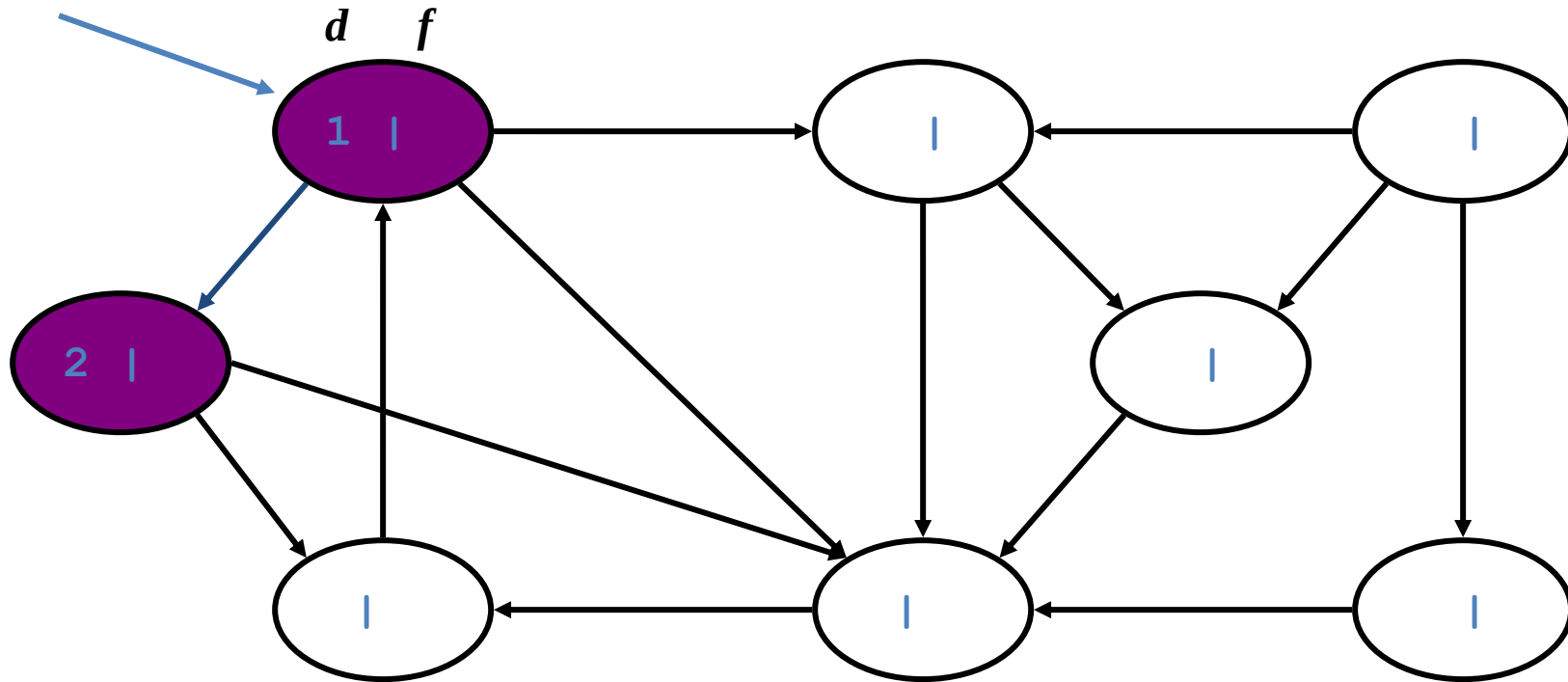
DFS Example

*source
vertex*



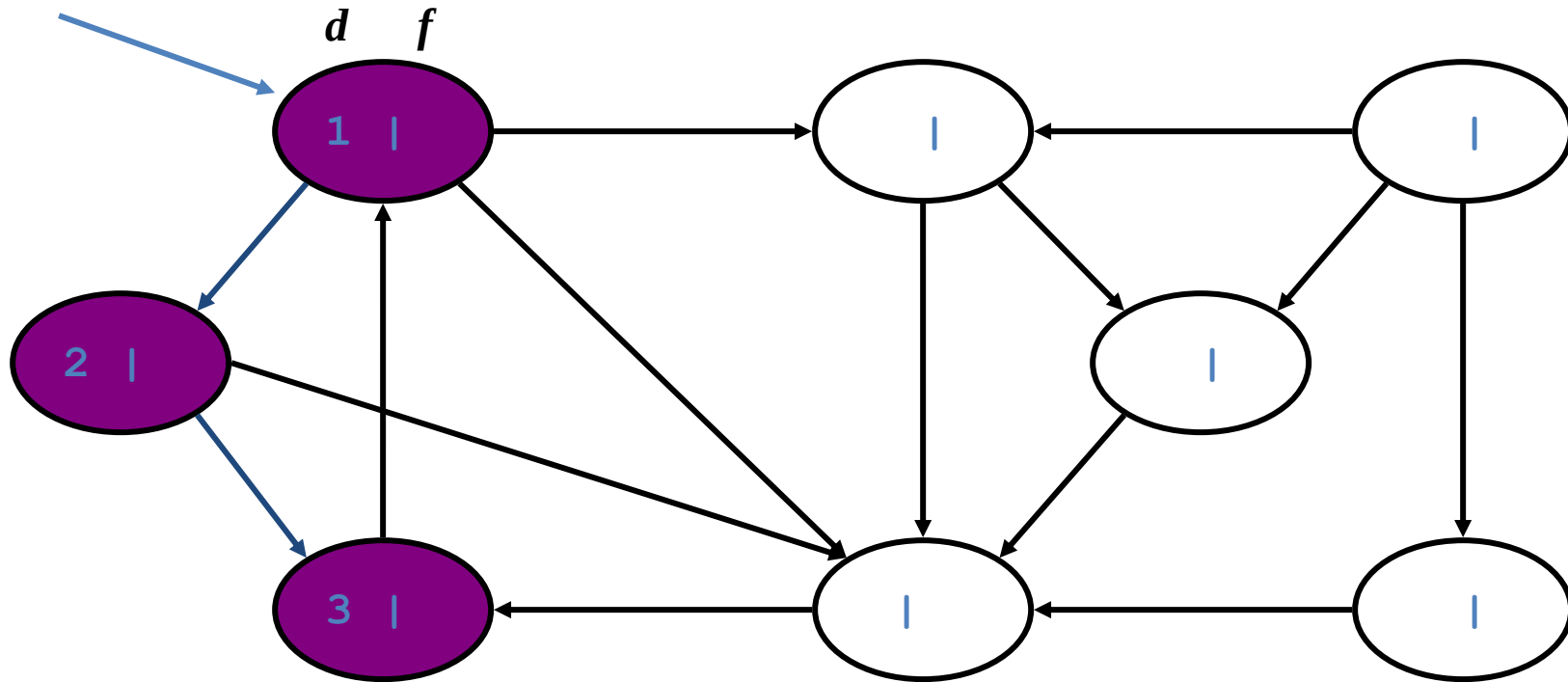
DFS Example

source
vertex



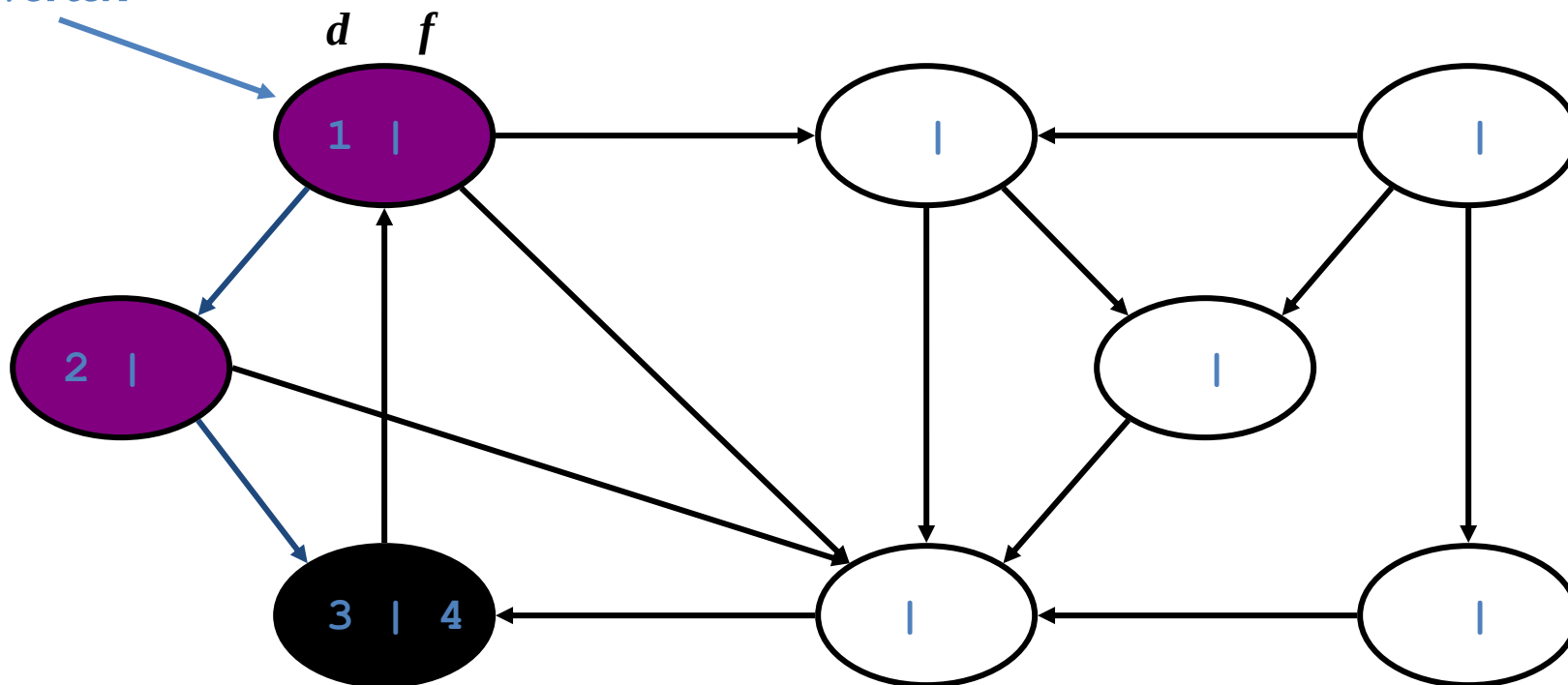
DFS Example

*source
vertex*



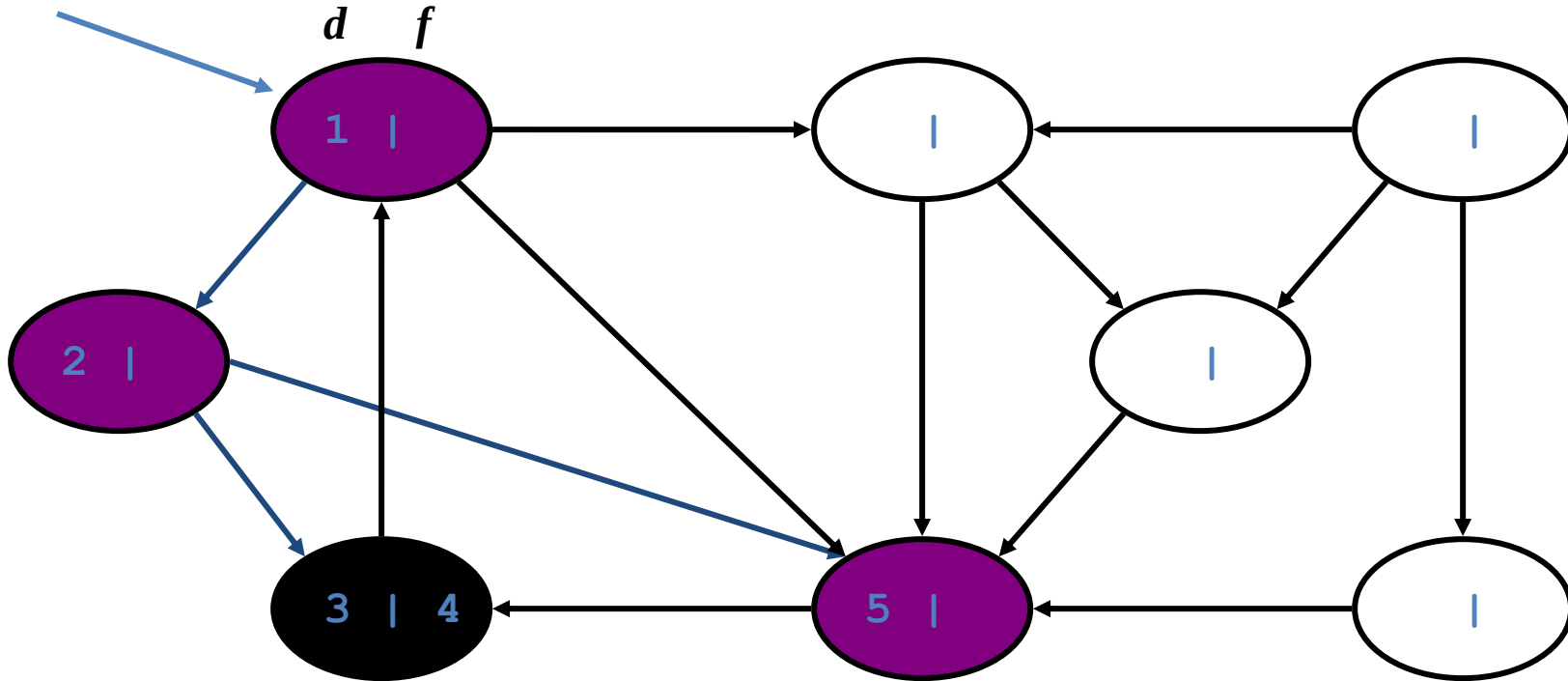
DFS Example

*source
vertex*



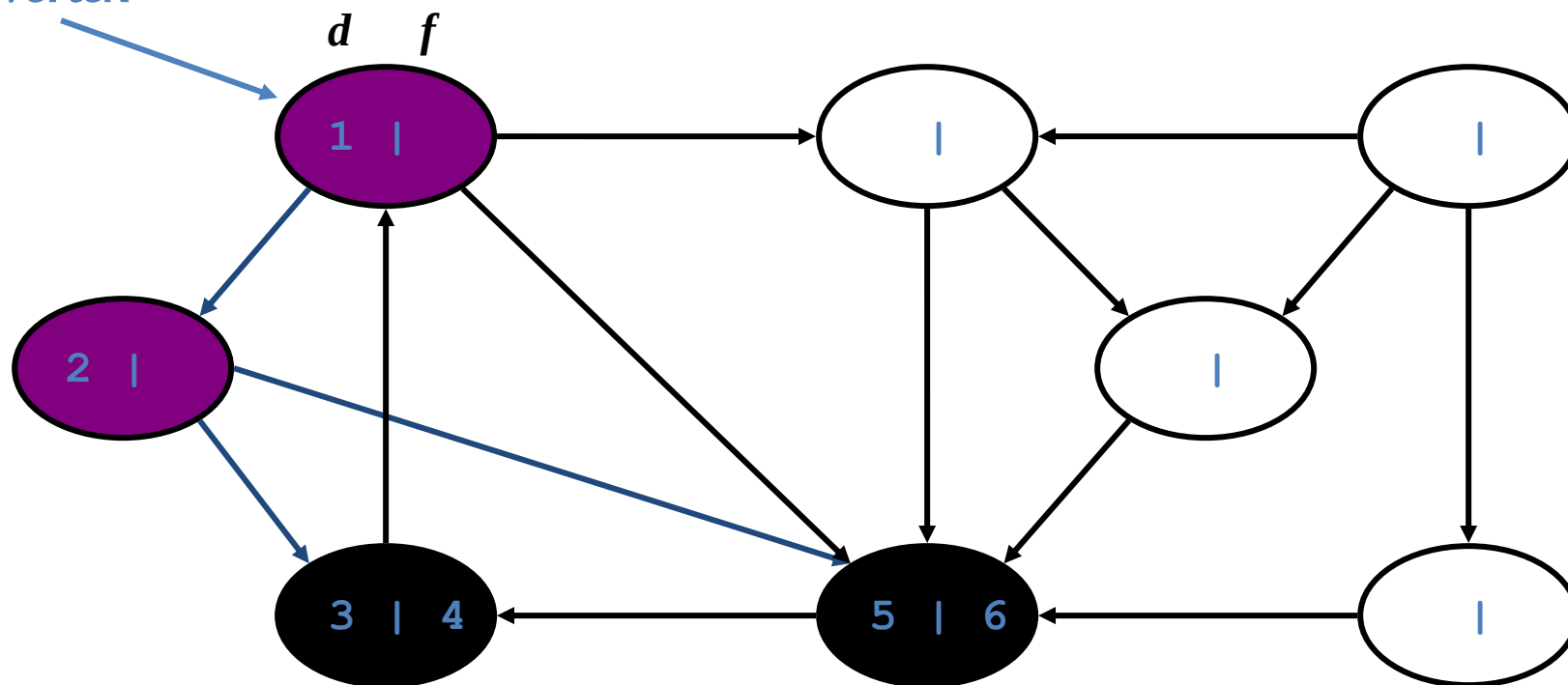
DFS Example

source
vertex



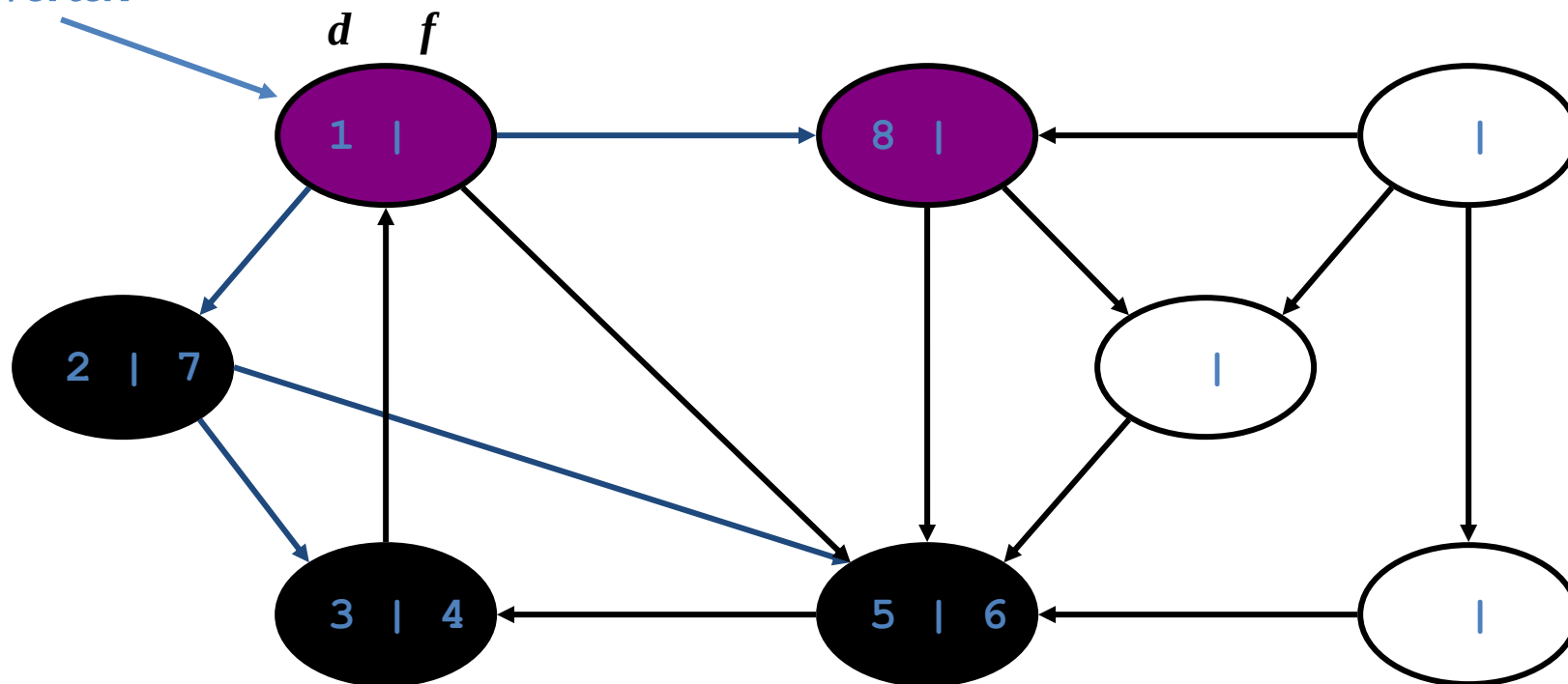
DFS Example

*source
vertex*



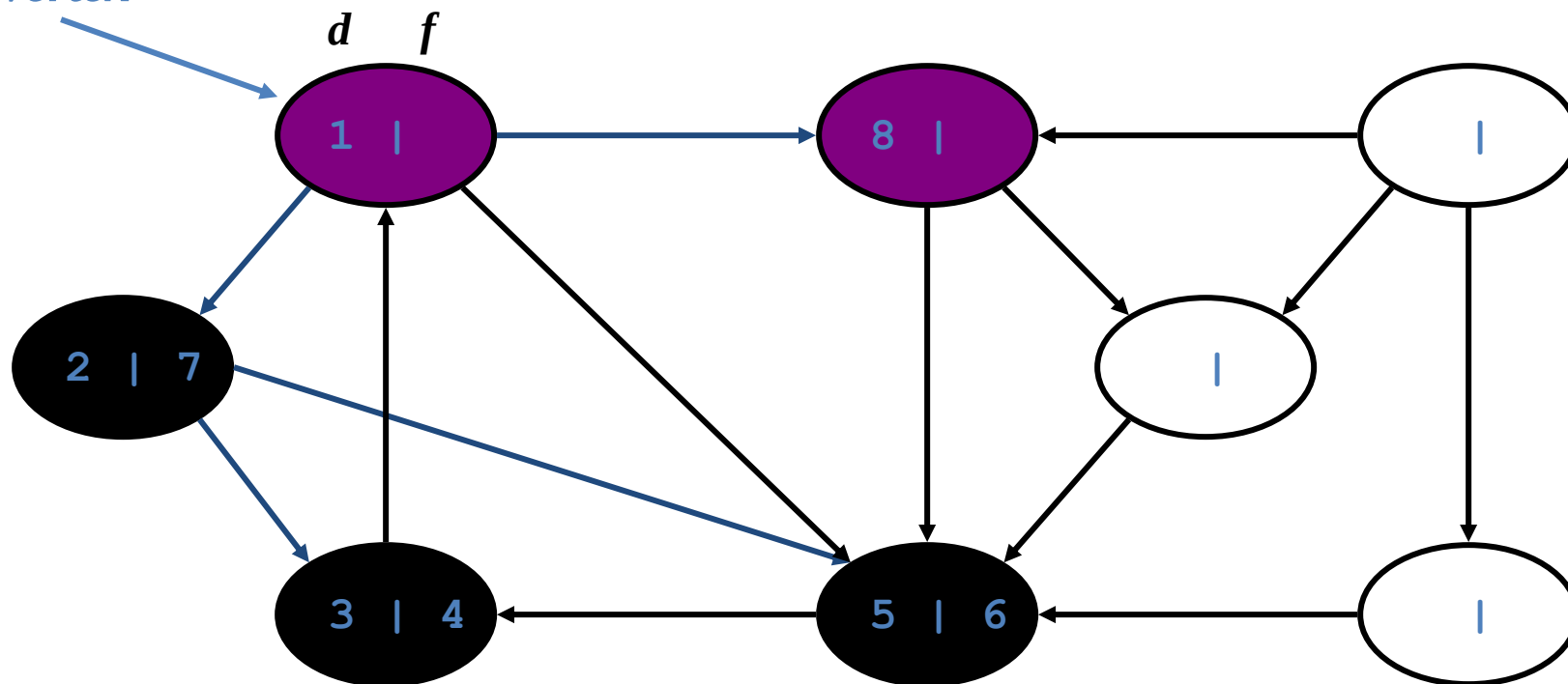
DFS Example

*source
vertex*



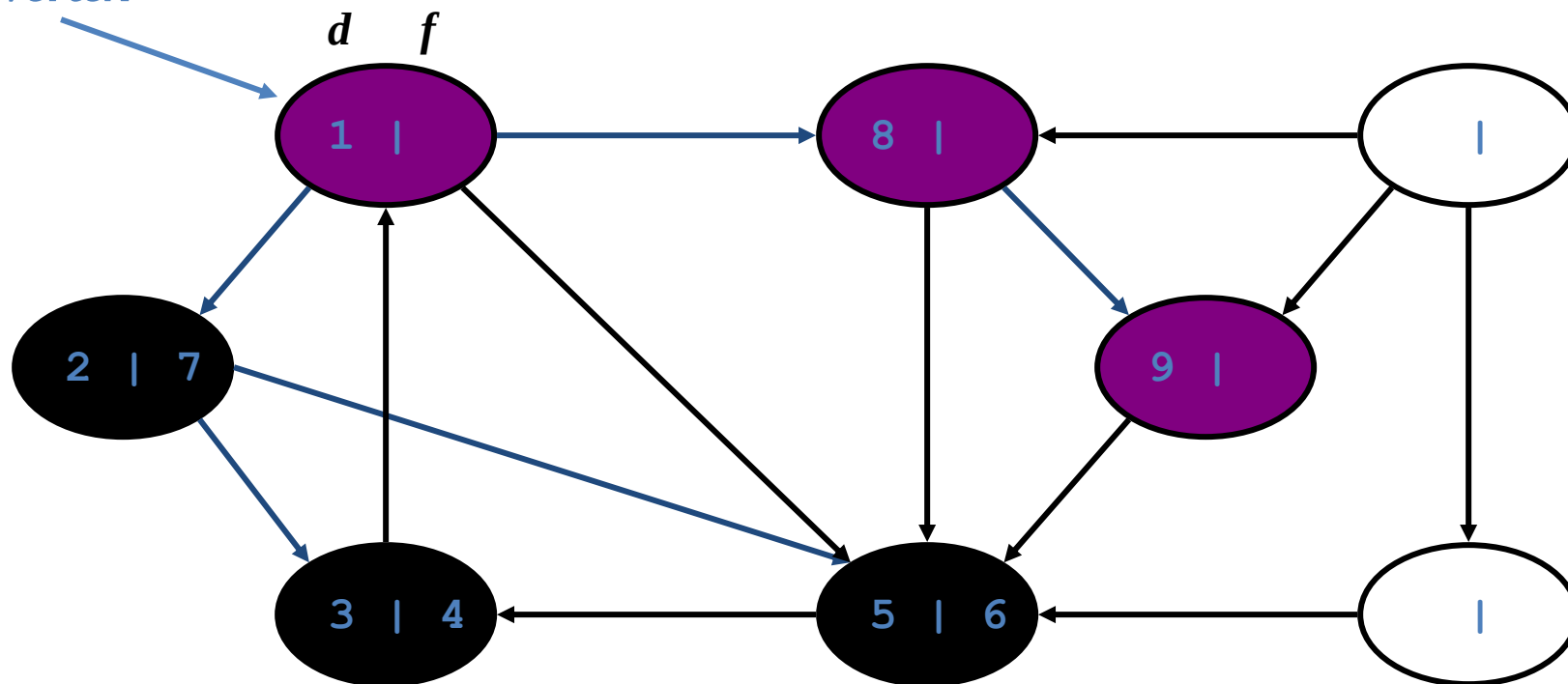
DFS Example

*source
vertex*



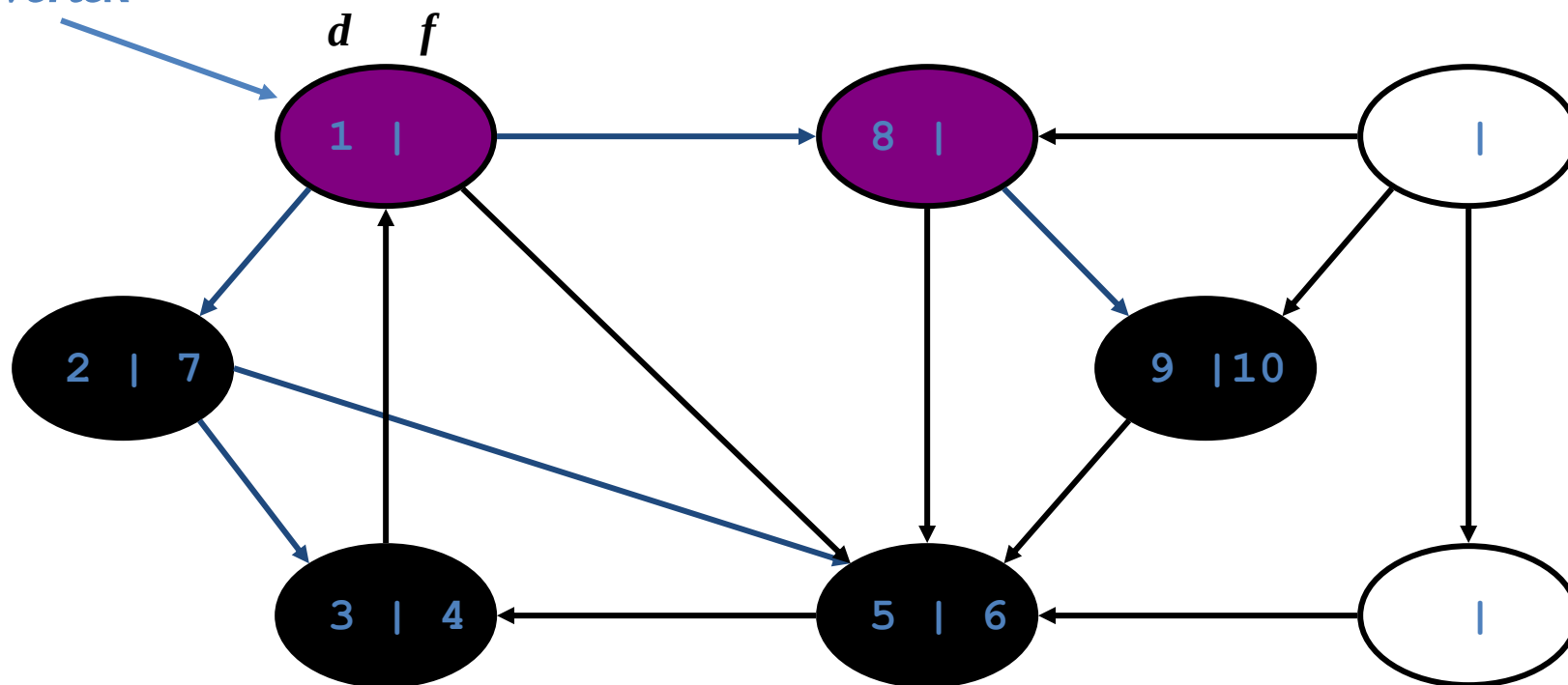
DFS Example

*source
vertex*



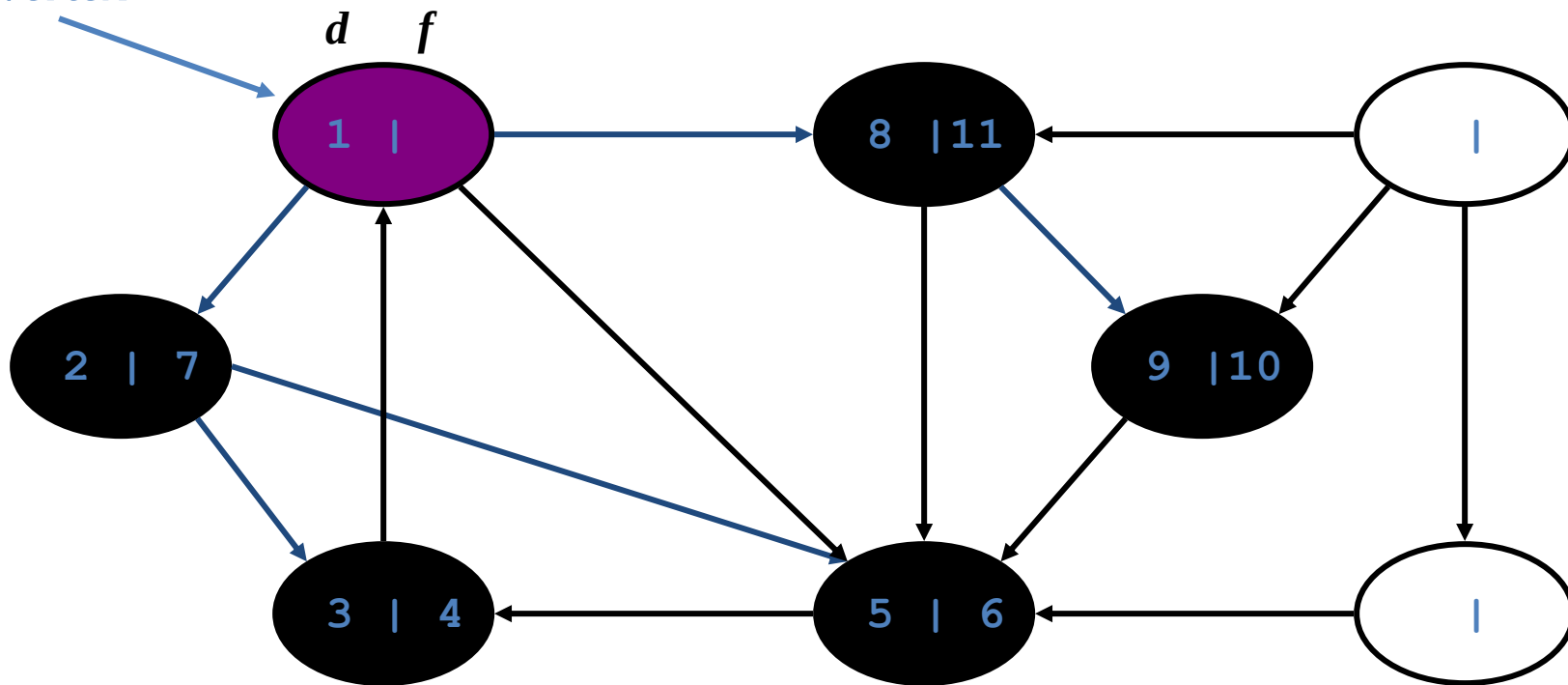
DFS Example

*source
vertex*



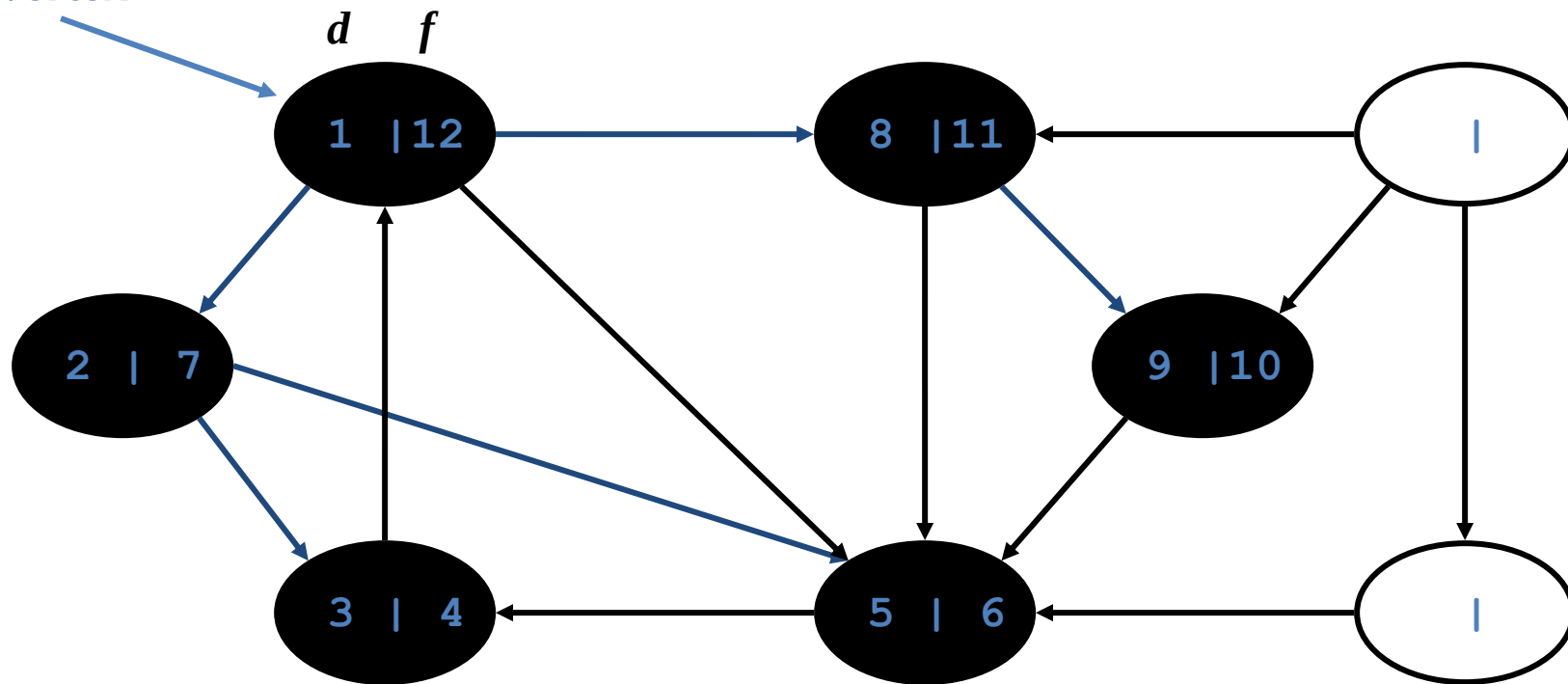
DFS Example

*source
vertex*



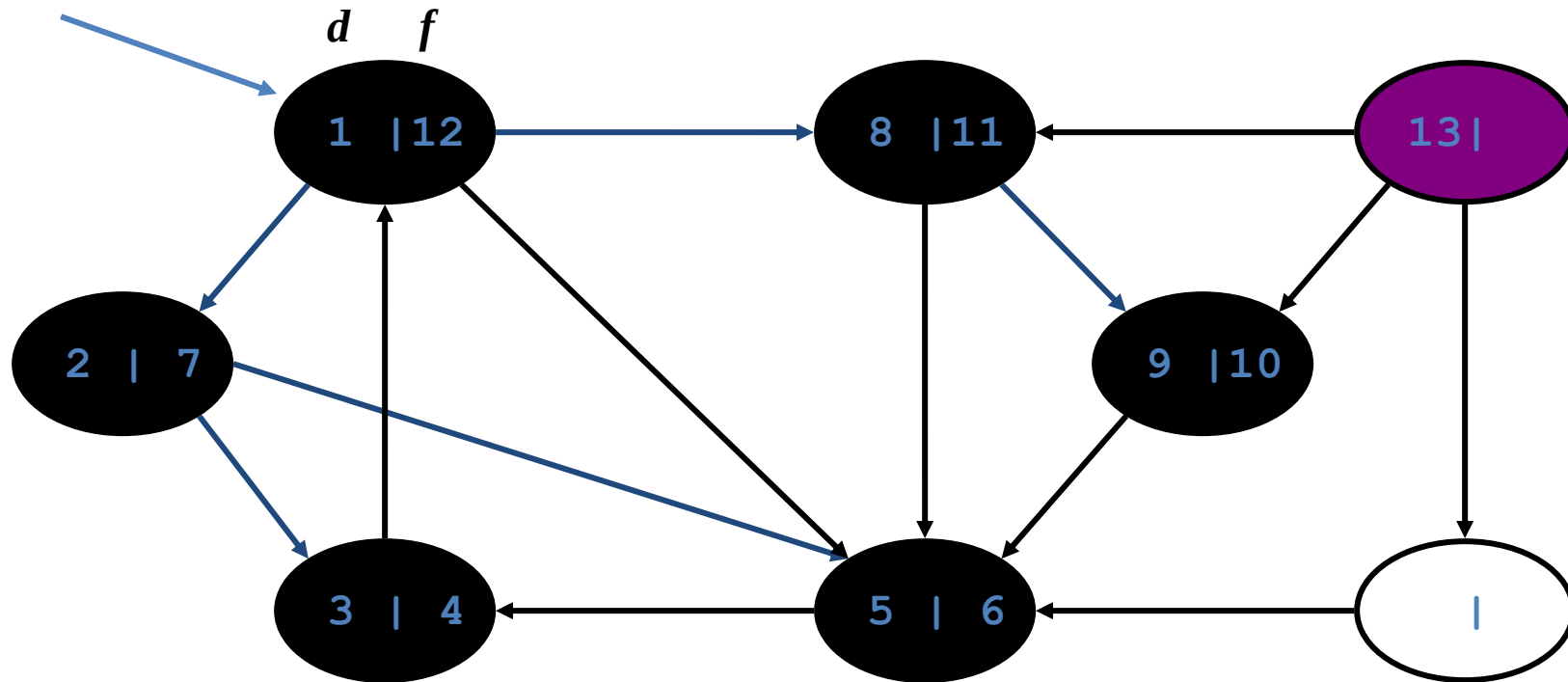
DFS Example

*source
vertex*



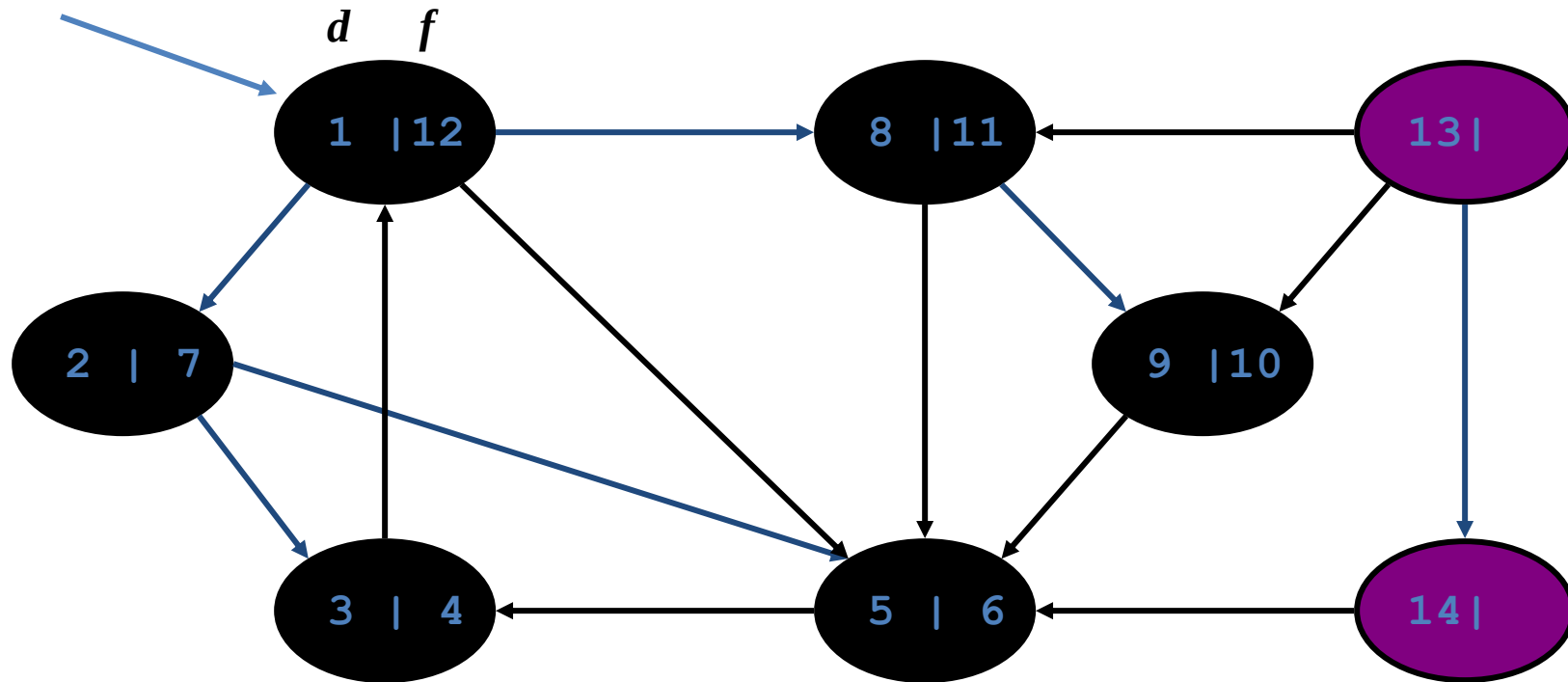
DFS Example

*source
vertex*



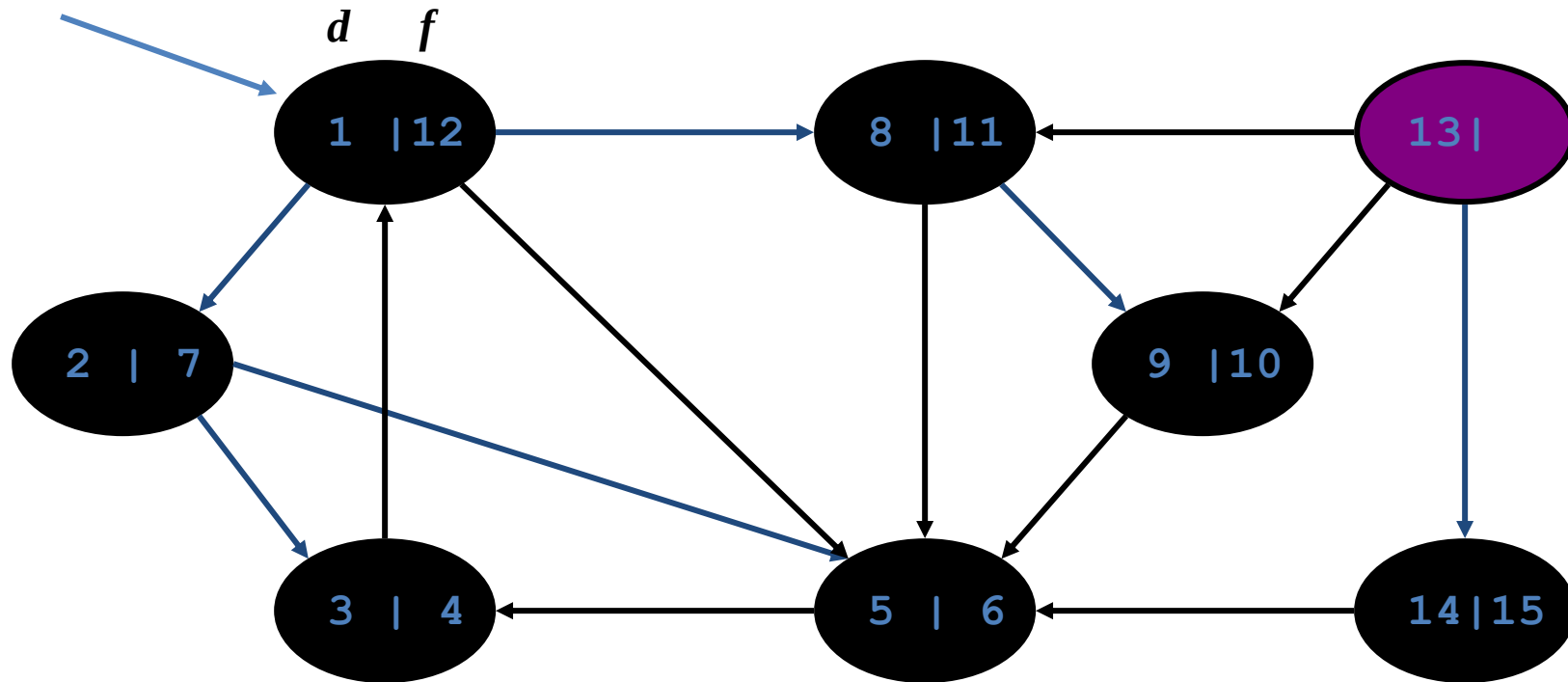
DFS Example

source
vertex



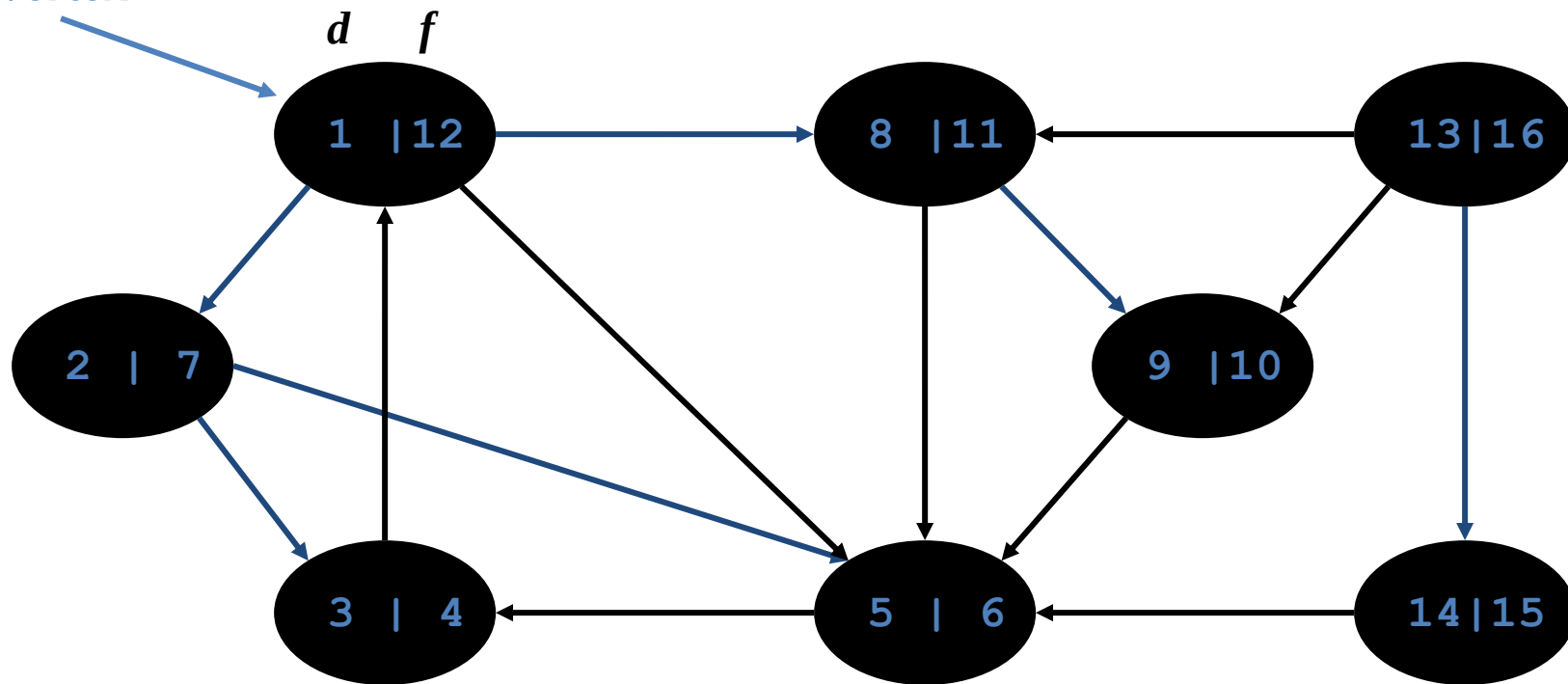
DFS Example

*source
vertex*



DFS Example

*source
vertex*

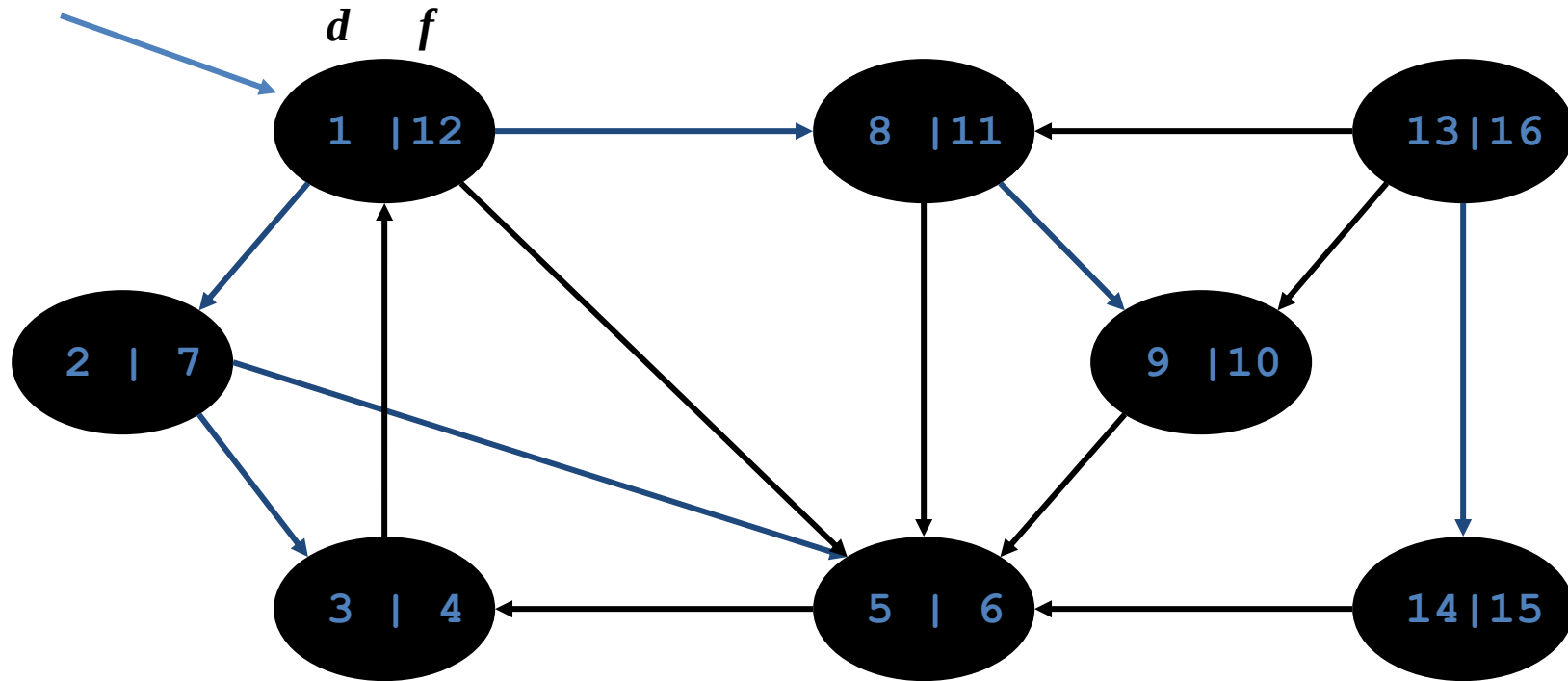


DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - The tree edges form a spanning forest
 - *Can tree edges form cycles? Why or why not?*

DFS: Kinds of edges

*source
vertex*



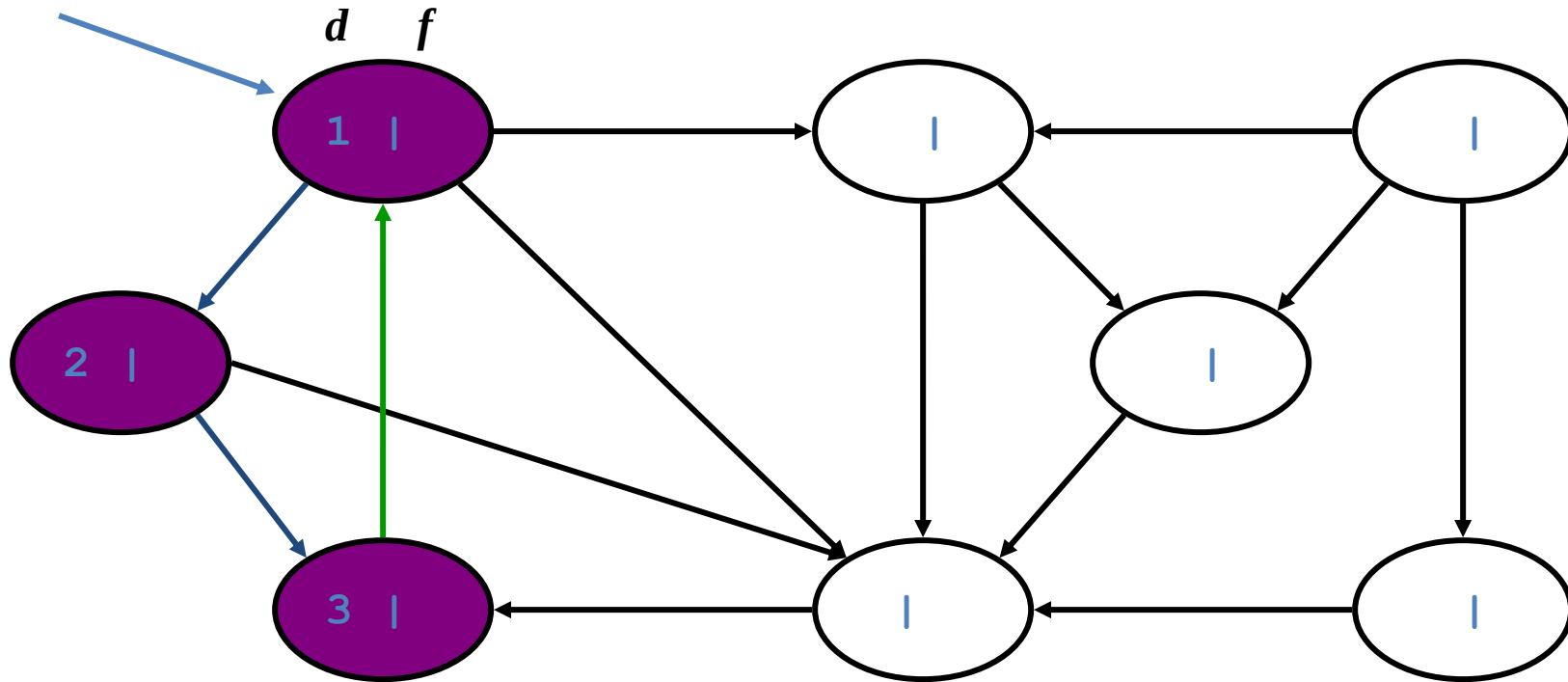
Tree edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - Encounter a grey vertex (grey to grey)

DFS: Kinds of edges

*source
vertex*



Tree edges

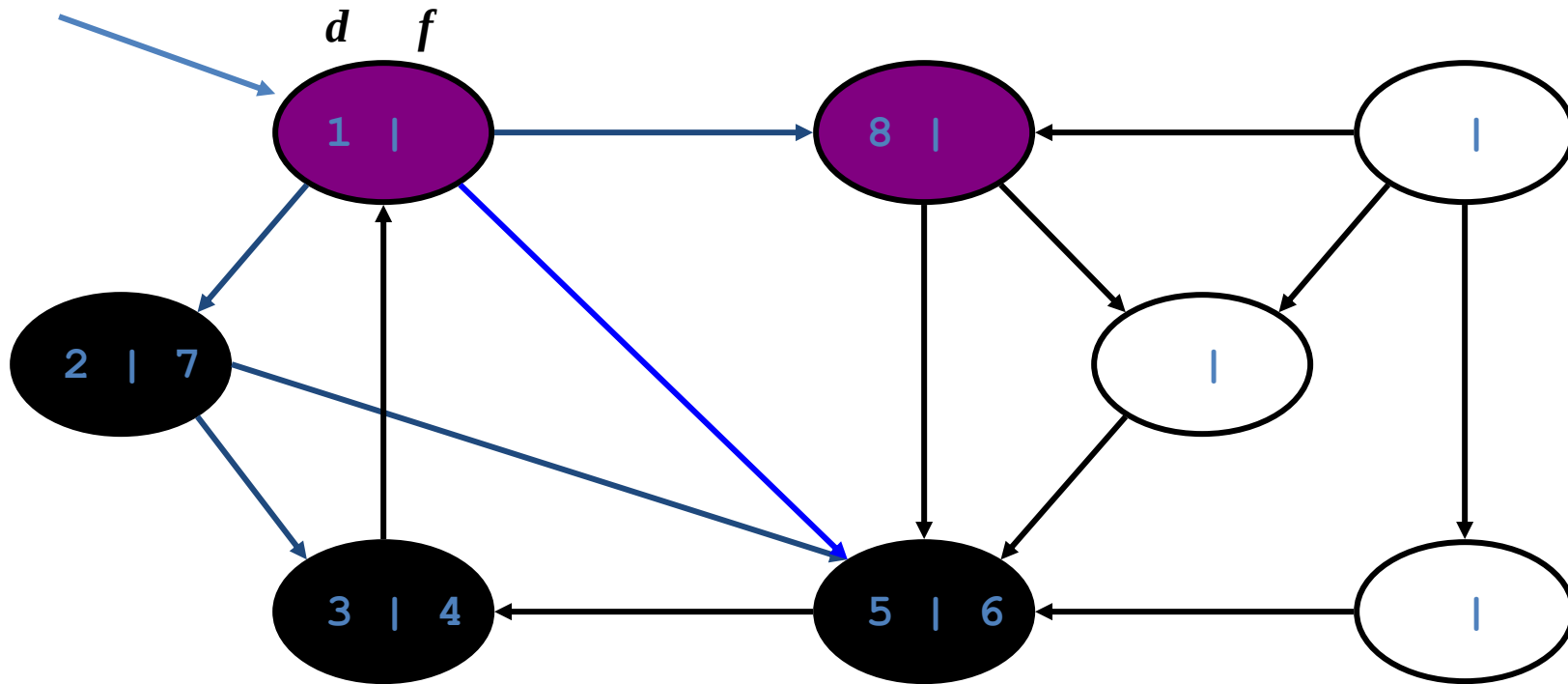
Back edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - Not a tree edge, though
 - From grey node to black node

DFS: Kinds of edges

*source
vertex*



Tree edges

Back edges

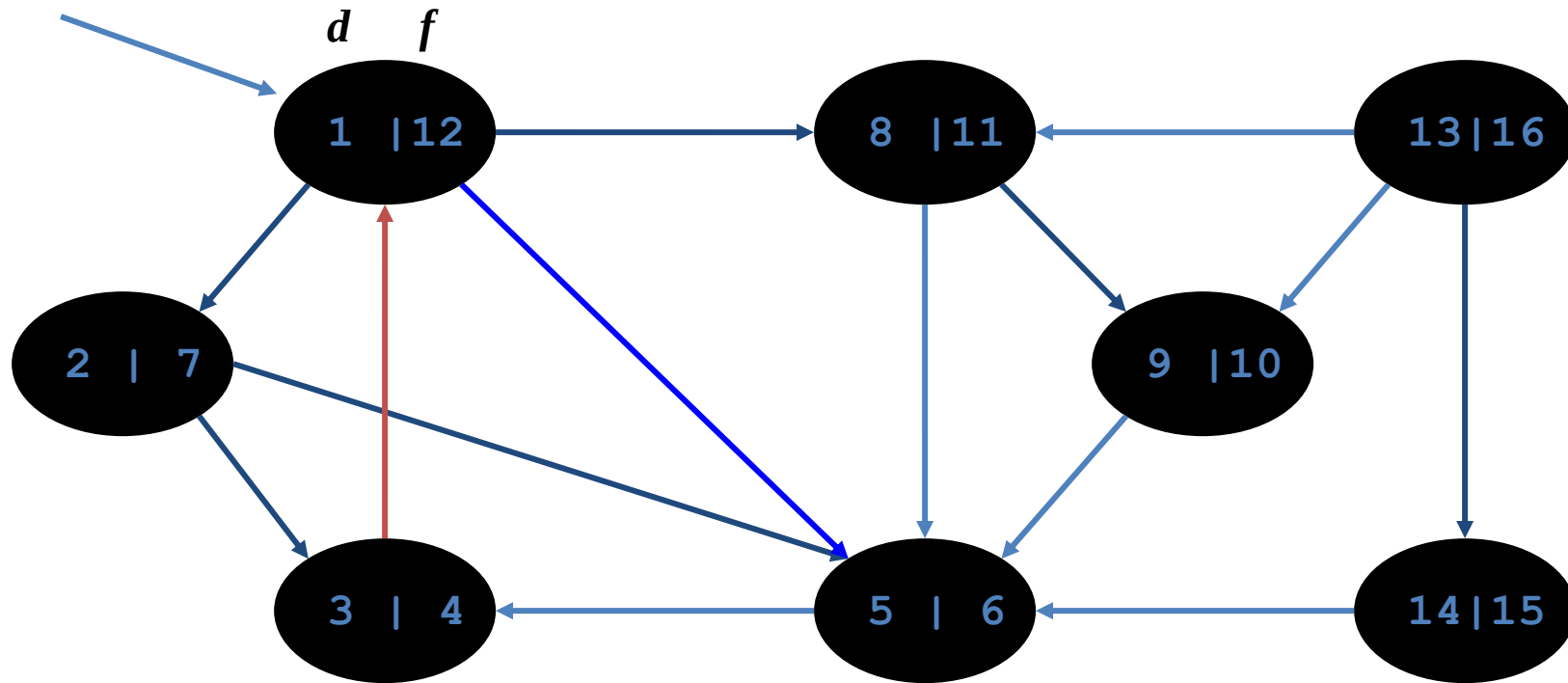
Forward edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - *Cross edge*: between a tree or subtrees
 - From a grey node to a black node

DFS: Kinds of edges

*source
vertex*



Tree edges

Back edges

Forward edges

Cross edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - *Cross edge*: between a tree or subtrees
- Note: tree and back edges are very important; some algorithms use forward and cross edges